TECHNICAL REFERENCE · JUNIOR EDITION

Azure for Junior AI Engineers

A practical handbook

The essential Azure services for LLM apps, RAG, agents, OCR pipelines and Python AI applications, with portal steps, Python examples and cost basics.

Mahmoud Refaey

LinkedIn: linkedin.com/in/mahmoudmalrefaey/

Reviewed: 13 October 2026

Primary language: Python · Audience: junior AI Engineers

Cloud services change continuously — always confirm against the linked official docs.

Contents

About This Guide . . . 5
- How to use this handbook . . . 5
- Scope and verification . . . 5
- Service finder . . . 7

Cloud Basics . . . 8
- Where your resources live . . . 8
- Three ways to work with a cloud . . . 9
- How your Python code signs in . . . 9
- Two kinds of permission . . . 10
- Regions and quotas . . . 10
- Names and tags . . . 10
- Start with the simplest option . . . 10

Service Map . . . 11
- Models and AI platform . . . 11
- Documents and OCR . . . 11
- Search and testing . . . 11
- Storage and databases . . . 11
- Running your app . . . 12
- Sign-in, keys and cost . . . 12

Models and the Foundry Platform . . . 13
- Microsoft Foundry . . . 13
- Foundry Models and Azure OpenAI . . . 15

AI Agents and Content Safety . . . 18
- Foundry Agent Service . . . 18
- Content Safety and guardrails . . . 20

OCR and Document Extraction . . . 22
- Document Intelligence . . . 22

Search and RAG . . . 24
- Azure AI Search . . . 24
- RAG basics: from documents to answers . . . 26
- Testing your answers . . . 29

Storage and Databases . . . 31

Blob Storage . . . 31
PostgreSQL and pgvector . . . 33
Cosmos DB . . . 34

Running Your Python App . . . 36
Azure Container Apps . . . 36
Azure Container Registry . . . 38
Azure Functions . . . 39

Sign-In, Roles and Secrets . . . 41
Managed identities and roles . . . 41
Azure Key Vault . . . 43

Monitoring Your App . . . 45
Azure Monitor and Application Insights . . . 45

Cost Basics and Safe Habits . . . 47
Cost, quotas and limits . . . 47

Four Starter Architectures . . . 49
Architecture 1: Chat with your documents (RAG) . . . 49
Architecture 2: AI agent app . . . 51
Architecture 3: OCR pipeline . . . 52
Architecture 4: Python AI API . . . 54
Which services go together? . . . 55

Learning Roadmap for Juniors . . . 56
Step 1: Cloud and Python basics (about 2 weeks) . . . 56
Step 2: Build with models and RAG (about 4 to 6 weeks) . . . 56
Step 3: Deploy, watch and control cost (about 3 to 4 weeks) . . . 56
Step 4: Agents and OCR (about 3 to 4 weeks) . . . 57
Certifications (optional) . . . 57

What to Learn Later . . . 58
Where to start when you are ready . . . 59

Quick Reference . . . 60
Which service should I use? . . . 60
Python packages and imports . . . 60
Ways to sign in . . . 61
Everyday commands . . . 61
Common errors and fixes . . . 61

Where to run your code . . . 62
What drives cost, and what to do about it . . . 62

Glossary . . . 63
Cloud basics . . . 63
AI and models . . . 63
Data and storage . . . 64
Sign-in and security basics . . . 64
Running and watching your app . . . 65

Coverage, Exclusions and Open Items . . . 66
What is covered . . . 66
Items not fully verified . . . 66
Exclusions . . . 66

Documentation Index . . . 67
Microsoft Learn documentation . . . 67
Microsoft pricing and tools . . . 71
SDK references and packages . . . 71
References: official sources . . . 72

About This Guide

It teaches the small set of cloud services you need to ship real projects: large language model (LLM) apps, retrieval-augmented generation (RAG) apps, AI agents, OCR pipelines and ordinary Python AI services.

Author: Mahmoud Refaey · LinkedIn: https://www.linkedin.com/in/mahmoudmalrefaey/

The guide is deliberately narrow. It leaves out enterprise security, complex networking, virtual machines, data engineering and large-scale architecture. Those topics matter later in a career, so they are listed in What to Learn Later instead of being taught here.

How to use this handbook

- **Read it in order the first time.** Start with Cloud Basics, then follow the chapters from models to deployment. Each chapter builds on the one before.
- **Use it as a reference afterwards.** Jump to a service from the Service Finder below, from the PDF bookmarks panel, or from the Quick Reference chapter.
- **Follow the official quickstarts for code.** This guide keeps only three short snippets: signing in with the Azure CLI, signing in from Python with `DefaultAzureCredential`, and a first model call. Each one matches a linked Microsoft page. Every other *Use it from code* section explains the key ideas in words and links to Microsoft's quickstart or SDK sample. Replace the `<placeholders>` with your own values. No real keys appear anywhere in this guide, and you should never paste a real key into code you share.

Every service section has the same five parts, so you always know where to look:

Part	What you will find
What it does	The service explained in plain words, with the few terms you must know.
When to use it	Good fits, bad fits and the nearest alternatives.
Set it up in the portal	Step-by-step Azure Portal (or Foundry portal) steps.
Use it from code	The key ideas in words, with links to Microsoft's quickstarts and SDK samples.
Main cost considerations	What you are charged for and how to avoid surprises.

Each section also ends with **common beginner mistakes** and links to the official documentation.

Scope and verification

> **Note — What was checked, and what was not**
>
> **Links:** on 12 and 13 October 2026 each external link was opened and its page title matched the topic. A working link does not prove that a claim is correct. **Claims:** on 13 October 2026 the Blob Storage tiers, Container Apps billing, Cosmos DB terminology, Foundry and Azure OpenAI resources, Agent Service concepts, Azure AI Search ingestion, the model sign-in scope, the Global Batch discount and the Prompt Shields API version were re-checked against the Microsoft Learn pages named in those sections. Other claims rest on the earlier review and have not all been re-checked. **Code:** only three short snippets remain, and each matches the Microsoft page cited under it. They were **not executed**. All other code is in the linked Microsoft quickstarts and SDK samples. Items we could not confirm are marked *Not verified*. Cloud platforms change often, so confirm anything that matters (prices, regional availability, quotas, preview status) in the linked documentation before you commit to it.

In scope. The services a junior AI Engineer needs for LLM apps, RAG, agents, OCR and Python AI applications: Microsoft Foundry and its models, the Foundry Agent Service, Content Safety, Document Intelligence, Azure AI Search, vector search in PostgreSQL, Blob Storage, PostgreSQL, Cosmos DB, Container Apps, Azure Functions, managed identities, Key Vault, monitoring and cost control.

Out of scope (by design). Everything listed in What to Learn Later, plus non-Python SDKs, absolute prices, and region-by-region availability tables. The full list of exclusions and of facts that could not be verified is in Coverage, Exclusions and Open Items.

Names you will meet in older material.

Older name (still common in blogs and courses)	Current name (October 2026)
Azure AI Studio, then Azure AI Foundry	**Microsoft Foundry** (portal at ai.azure.com)
Azure AI Services / Cognitive Services	**Foundry Tools** (Document Intelligence, Content Safety, Speech, Vision, Language and others)
Azure OpenAI Service as its own resource	Azure OpenAI models are deployed inside a **Foundry resource**
Assistants API, threads and runs	**Responses API** with conversations; Foundry agents use agent versions
Azure Active Directory (Azure AD)	**Microsoft Entra ID**

Python and package versions. Python 3.10 or later is assumed. Pin the package versions in your own projects, because minor releases can rename preview features.

Package (PyPI)	Version assumed	Used for
`azure-identity`	latest 1.x	Signing in with `DefaultAzureCredential` and managed identities
`azure-ai-projects`	>= 2.3.0	Foundry project client: agents and an OpenAI client factory. 2.x is not compatible with 1.x code.
`openai`	latest	Calling models through the Responses and Chat Completions APIs
`azure-search-documents`	>= 12.0.0	Azure AI Search
`azure-ai-documentintelligence`	1.0.2	OCR and document extraction
`azure-ai-contentsafety`	1.0.0	Checking text for harmful content
`azure-ai-evaluation`	1.18.x	Scoring answer quality
`azure-storage-blob`, `azure-cosmos`, `azure-keyvault-secrets`	latest stable	Storage, database and secrets
`azure-monitor-opentelemetry`	latest stable	Sending traces to Application Insights

Conventions. Placeholders are written in angle brackets (`<resource-name>`) or as UPPER_SNAKE_CASE environment variables (`AZURE_SEARCH_ENDPOINT`). Portal paths look like **Resource › Settings › Networking**. Portals change often, so where a button might move the guide describes what to set, not just where to click. Prices are never quoted as fixed numbers; each section lists what you are charged for and links to the official pricing page.

Callouts used in this guide.

> **Best practice**
> A recommended habit.

> **Warning**
> A common mistake that breaks things.

> **Cost risk**
> A setting that can create unexpected charges.

> **Security**
> A basic safety habit for keys, access and data.

> **Not fully verified**
> Information that could not be confirmed from official sources. Check it before you rely on it.

Service finder

Click a service name to jump to its section. The Service Map lists every service in one place.

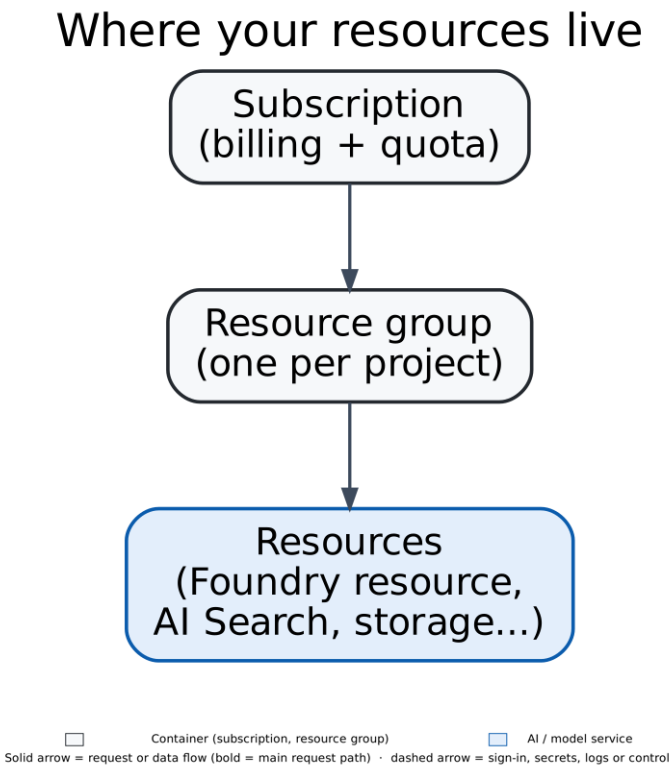
What you want to do	Service
Call an LLM from Python	Microsoft Foundry and Foundry Models
Build an AI agent	Foundry Agent Service
Block harmful or unsafe content	Content Safety
Read text from PDFs and scans (OCR)	Document Intelligence
Search your documents for RAG	Azure AI Search
Build and test the RAG pipeline	RAG basics and Testing answers
Store files	Blob Storage
Store app data and vectors	PostgreSQL and pgvector
Store chat history	Cosmos DB
Host a Python API	Container Apps
Run small event-driven code	Azure Functions
Sign in without passwords	Managed identities
Store a secret safely	Key Vault
See errors, latency and tokens	Monitoring
Control spending	Cost basics

Cloud Basics

Before you learn individual services, you need a small mental model of how a cloud is organised and how your code proves who it is. Most problems that stop junior engineers are not about the AI model. They are about permissions, regions, quotas and settings. This chapter gives you the vocabulary the rest of the guide relies on.

Where your resources live

Everything you create in a cloud is a **resource**: a model deployment, a database, a storage account. Azure groups resources so you can see what you own, who pays for it and how to delete it.



On Azure, resources live in a resource group inside a subscription.

Idea	What Azure calls it
The thing that is billed	**Subscription**
A folder for related resources	**Resource group**. Deleting the group deletes everything inside it, which makes cleanup easy.
Where a resource runs	A **region** you pick when you create it
A resource's full name	Resource ID: `/subscriptions/<id>/resourceGroups/<rg>/providers/...`

> **Best practice — Keep one resource group per project**
> Put everything for one project and environment (for example `ragbot-dev`) in one Azure resource group. When you finish experimenting, delete the group and you stop paying for everything inside it.

Three ways to work with a cloud

Way	Tool
Web portal (click and see)	Azure Portal (portal.azure.com) and the **Foundry portal** (ai.azure.com)
Command line	Azure CLI (`az`)
Python code	Separate `azure-*` packages that share `azure-identity`

The portal is best for learning and exploring. The CLI is best for repeatable steps. The Python SDK is what your application uses. This guide shows the portal steps and links to Microsoft's quickstarts for code.

Sign in from your terminal, create a resource group for a project, and delete the group when you are done. `az login` opens a browser to sign you in.

```Shell (bash)
az login
az account set --subscription "<subscription ID or name>"
az group create --name <resource-group> --location <region>
az group delete --name <resource-group>
```

Source: Sign in with Azure CLI and Manage Azure resource groups by using Azure CLI

How your Python code signs in

Your code must prove who it is before a cloud accepts its requests. Azure does this with a **credential chain**: the SDK tries several places in order and uses the first that works. The same code then runs on your laptop and in the cloud without changes.

Create one credential and pass it to every client. Use `DefaultAzureCredential` for local development: on your laptop it uses your `az login`.

```Python
from azure.identity import DefaultAzureCredential
from azure.storage.blob import BlobServiceClient

credential = DefaultAzureCredential()
blob_service_client = BlobServiceClient(
    account_url="https://<my_account_name>.blob.core.windows.net",
    credential=credential)
```

Source: Credential chains in the Azure Identity library for Python

In deployed apps, prefer `ManagedIdentityCredential` (with `AZURE_CLIENT_ID` for a user-assigned identity), or set the environment variable `AZURE_TOKEN_CREDENTIALS=prod` so `DefaultAzureCredential` only tries production credentials. Either way, the app signs in with the identity attached to the service that runs your code (see Managed identities).

> **Security — Never put keys in your code**
>
> Do not paste API keys, access keys or connection strings into source files, notebooks or Git. Prefer signing in with an identity, as shown above. When you must use a key, keep it in Key Vault and read it at run time. If a key is ever committed to Git, treat it as stolen and replace it.

Two kinds of permission

Every cloud service has two kinds of access, and mixing them up causes most "403 Forbidden" errors.

- **Managing the resource** means creating, changing or deleting it. On Azure this needs roles such as *Contributor*.
- **Using the resource** means calling the model, querying the index or reading a blob. On Azure this often needs a separate role such as *Foundry User*, *Search Index Data Reader* or *Storage Blob Data Reader*.

> **Warning — Owner is not always enough**
> You can be *Owner* of an Azure subscription, create an AI Search service and still get **403** when you query it with `DefaultAzureCredential`. Assign yourself the right *using* role (for example *Search Index Data Contributor*) and wait a few minutes for it to take effect.

Regions and quotas

AI services are not available in every region, and new models usually appear in a few regions first. Before you build, check three things:

1. **Is the service and the exact model available in my region?** Each service section links to the availability page.
2. **Do I have enough quota?** A **quota** is a limit on how much you may use. Azure measures model quota in tokens per minute (TPM) and requests per minute (RPM) for each model, deployment type, subscription and region.
3. **Where may my data be processed?** Some deployment types may process requests in any region. If your data must stay in a geography, choose the matching option (see Foundry Models).

> **Warning — A quota error is not an outage**
> HTTP **429** means "too many requests". It usually means you reached your quota. Retry with a short wait, send fewer requests at once, or ask for more quota.

Names and tags

Choose a naming pattern on day one, for example `<project>-<service>-<env>`, such as `ragbot-search-dev`. Some resources (storage accounts, container registries) need globally unique lowercase names, so add a short suffix of your own. Add the tags `project` and `owner` to everything. Tags let you find resources and see what each project costs.

Start with the simplest option

Azure offers the same capability at several levels of effort. As a junior engineer, always start at the top of this table and only move down when you have a clear reason.

Level	Example	Choose when
Ready-made API	Foundry Models, Document Intelligence	You need a capability, not a model you control. Fastest to ship and least to maintain.
Managed hosting for your code	Container Apps, Azure Functions	You write your own logic but do not want to manage servers.
Servers and clusters	Virtual machines, Kubernetes	You need full control. This is a later step; see What to Learn Later.

Service Map

Every service in this guide on one page. Click a name to open its section. Official documentation links are in each section.

Models and AI platform

Service	What it does	Use it for	Docs
Microsoft Foundry	Where you deploy models, build agents and manage access.	Always: it is the home of your AI work.	Learn
Foundry Models (Azure OpenAI)	Pay-per-token chat, embedding and other models.	Any LLM, RAG or agent app.	Learn
Foundry Agent Service	Runs agents that choose tools step by step.	The model must take actions or use tools.	Learn
Content Safety and filters	Blocks harmful content and prompt-injection attacks.	Any app real users can reach.	Learn

Documents and OCR

Service	What it does	Use it for	Docs
Document Intelligence	Reads text, tables and fields from scans and PDFs.	OCR pipelines and RAG over scanned files.	Learn

Search and testing

Service	What it does	Use it for	Docs
Azure AI Search	Indexes content; finds chunks by keyword, meaning or both.	RAG retrieval.	Learn
RAG basics	The steps from documents to grounded answers.	Chat with your documents.	Learn
Testing your answers	Measures retrieval and answer quality.	Before every release.	Learn

Storage and databases

Service	What it does	Use it for	Docs
Blob Storage	Stores files such as PDFs and images.	Source documents, scans, results.	Learn
PostgreSQL with pgvector	A database that also searches embeddings.	Vectors next to app data.	Learn
Cosmos DB	A fast NoSQL database.	Chat history and app state.	Learn

Running your app

Service	What it does	Use it for	Docs
Container Apps	Runs your container without servers to manage.	Python AI APIs.	Learn
Container Registry	Stores your container images.	Whenever you deploy a container.	Learn
Azure Functions	Runs small code when an event happens.	Reacting to uploads, small tools.	Learn

Sign-in, keys and cost

Service	What it does	Use it for	Docs
Managed identities and roles	Sign-in without passwords, with limited permissions.	Every cloud-to-cloud call.	Learn
Key Vault	Stores secrets such as third-party API keys.	Secrets you cannot avoid.	Learn
Monitor and Application Insights	Logs, metrics, traces and alerts.	Before the first deployment.	Learn
Cost, quotas and limits	Shows spend, sends alerts, limits usage.	Set a budget on day one.	Learn

Models and the Foundry Platform

This chapter covers the first thing every AI Engineer does: call a large language model (LLM) from Python. On Azure you do this through **Microsoft Foundry**. You will learn how to create the resources, deploy a model, and write the code that talks to it.

Microsoft Foundry

What it does

Microsoft Foundry (formerly Azure AI Studio and Azure AI Foundry) is the Azure place to deploy models, build agents, test them and connect them to your data. You work with three things:

- **Foundry resource.** The main Azure resource. It has an endpoint, holds your **model deployments**, and is where access is controlled. Microsoft's overview describes it as a single resource with projects, instead of the earlier combination of a hub, an Azure OpenAI resource and an Azure AI Services resource.
- **Project.** A workspace inside the Foundry resource, usually one per application. Agents, files and connections live in a project. Each project has its own endpoint: `https://<resource-name>.services.ai.azure.com/api/projects/<project-name>`.
- **Model deployment.** A named copy of a model that your code can call. You choose the model, a deployment type and how much capacity it gets.

> **Note — Older resources still exist**
> You may meet an existing **Azure OpenAI resource** or a **hub-based project** at work or in older tutorials. Microsoft documents an optional, opt-in upgrade of an Azure OpenAI resource to a Foundry resource that keeps its endpoint, keys and existing state, and says both are supported before and after the upgrade. Upgrading has prerequisites (for example the Owner role) and limits, so read Upgrade from Azure OpenAI to Foundry first. This guide shows the Foundry resource because it is the current recommended starting point; it does not claim that older resources must be migrated.

Your code usually does two things with Foundry: create an `AIProjectClient` for the project (used for agents) and an OpenAI-style client for the model (used to send prompts).

When to use it

Use Foundry when you want to call models hosted on Azure, build agents, or keep your AI work in one Azure resource. If you only need a single model, you still create a Foundry resource and a deployment, but you may never use projects much.

Foundry is not a place to train your own models from scratch. That is a later topic (see What to Learn Later).

Set it up in the portal

Before you start: an Azure subscription, a resource group, permission to create resources (*Contributor* on the resource group), and a region that offers the model you want.

1. **Create the resource.** In the Azure Portal search for *Foundry* and choose **Create**. You can also start from the Foundry portal at `ai.azure.com`, which creates the resource and a first project for you. Pick the subscription, resource group, a unique name (it becomes part of the endpoint), the region and the project name.
2. **Leave networking at the default while you learn.** Private networking is an advanced topic, covered under What to Learn Later.
3. **Give yourself access.** Open the resource, choose **Access control (IAM)** and add the role **Foundry User** for your user. This is the role that lets you *use* models and agents.

4. **Open the Foundry portal.** From the resource choose **Go to Foundry portal**.
5. **Deploy a model.** In the model catalog choose a model, select **Deploy**, give the deployment a name, pick a deployment type (for example *Global Standard*) and set the capacity in tokens per minute. See Foundry Models for what the types mean.
6. **Test it.** Open the deployment in the **playground**, send a prompt, and use **View code** to copy a working example.
7. **Clean up.** Delete deployments and the resource group when you are done. A deleted Foundry resource is only *soft-deleted*. To reuse the same name you must **purge** it (portal: **Manage deleted resources**; CLI: `az cognitiveservices account purge`).

If something fails: **401** usually means a wrong or missing sign-in; **403** means a missing role; **404 DeploymentNotFound** means the deployment name in your code does not match the portal; **429** means you reached your quota.

Task	How to do it
Create a place to work	Foundry resource and project
Make a model callable	Create a deployment
Let a developer use models	Role **Foundry User**

Use it from code

Your code talks to Foundry in two ways: a **project client** (`AIProjectClient` from the `azure-ai-projects` package) for project features such as agents, and an **OpenAI client** for model calls (see Foundry Models). Microsoft's Quickstart: Get started with Microsoft Foundry SDK shows how to install the packages, set the project endpoint and make a first call.

Key points:

- Keep the endpoint in configuration (an environment variable), never a key.
- Use `azure-ai-projects` 2.x. Microsoft's quickstart notes that 2.x code does not work with 1.x, which many older tutorials use.
- Create a client once and reuse it. Set a timeout and a retry limit so a slow call does not hang your app.
- To repeat a setup, create the resource, project and deployment with the Azure CLI (`az cognitiveservices`; see the CLI reference) or with the Bicep quickstart. Use `az cognitiveservices model list --location <region>` to see which models you can deploy.

Main cost considerations

The Foundry resource and projects are free. You pay for what runs inside them: model **tokens** (small pieces of text that models read and write), other Foundry Tools, and connected services such as AI Search or storage. Quota is a separate limit and costs nothing by itself.

> **Cost risk — Delete what you do not use**
> Pay-per-token deployments cost nothing while idle. Some deployment types (provisioned throughput) are billed by the hour even with no traffic. Use pay-per-token types while you learn and delete test deployments.

> **Best practice — Tag and measure**
> Tag the Foundry resource with the project name, and log the token usage of every response (the `usage` field). Billing alone cannot tell you which feature used the tokens.

Common beginner mistakes

- Assuming that being subscription *Owner* is enough to call a model. You also need the **Foundry User** role.
- Following old tutorials that use `azure-ai-projects` 1.x, `azure-ai-inference`, `AzureOpenAI()` with monthly `api-version` values, or threads and runs. That is the previous generation.
- Creating one Foundry resource for everyone. Use one resource per project and environment so quota and access stay easy to reason about.
- Pasting an API key into code. Sign in with `DefaultAzureCredential` instead.

Official documentation

- What is Microsoft Foundry?
- Quickstart: get started with code (azure-ai-projects 2.x)
- Built-in RBAC roles for Microsoft Foundry
- Create Foundry projects
- Azure CLI reference: az cognitiveservices account

Foundry Models and Azure OpenAI

What it does

This is the part of Foundry that runs the model. You send text (and sometimes images) and get text, structured data, tool calls or **embeddings** back. An embedding is a list of numbers that captures the meaning of a piece of text, and it is what makes search by meaning possible.

On Azure you call a model through a **deployment**. The deployment has a *name* (what you pass as `model` in code), a *model and version*, a *deployment type* and a *capacity*.

Deployment types you will meet first:

Deployment type	Where data is processed	Use it when
Global Standard	Any Azure region	The default for most projects. Pay per token, highest quota.
Data Zone Standard	Inside a data zone (US, EU or Asia Pacific)	You need processing to stay in a zone. Pay per token.
Standard	In the region of your deployment	You need processing to stay in one region.
Global Batch	Any Azure region	Large offline jobs where a 24-hour turnaround is fine, at a 50% discount compared with Global Standard.

For every deployment type, data stored at rest stays in the Azure geography you chose; the types differ in where requests are processed.

Not every model supports every type. Check the model's page in the catalog. Other types (provisioned throughput) are a later topic.

Which API to use. Microsoft recommends the OpenAI **v1 API** at `https://<resource>.openai.azure.com/openai/v1/`, used with the normal `OpenAI()` client from the `openai` package. The v1 API also accepts `https://<resource>.services.ai.azure.com/openai/v1/` as the base URL. It supports the **Responses API** (recommended for new work) and the older **Chat Completions** API (many frameworks use it), plus embeddings. For sign-in without keys the token scope is `https://ai.azure.com/.default`.

What every AI Engineer should know how to do:

- Write clear **instructions** (a system prompt) and keep untrusted text, such as retrieved documents, clearly separated from them.

- Ask for **structured output** so the model returns JSON your code can trust.
- Use **tool (function) calling**: the model asks for a function to run, and *your code* runs it.
- Create **embeddings** for search.
- **Stream** the answer so users see text as it is produced.
- Handle errors and rate limits (see below).

When to use it

- Start with **Global Standard**. Move to Data Zone or Standard only when your data rules require it.
- Use **Batch** for work that can wait, such as classifying thousands of old tickets.

Set it up in the portal

Before you start: a Foundry resource, the *Foundry User* role, and quota for the model you want.

1. In the Foundry portal (`ai.azure.com`) open your project and then the model catalog. Filter by what you need (chat, reasoning, embeddings, image).
2. Open a model's card and read its context window, supported types and regions, and its retirement date.
3. Select **Deploy**. Give the deployment a purpose-based name such as `chat-primary` or `embed-small`, choose the deployment type and the capacity (tokens per minute).
4. Keep the default **content filter** (see Content Safety) while you learn.
5. Deploy it, then test it in the **playground**. Try different instructions and use **View code** for a snippet.
6. Check usage in the resource's **Metrics** page: requests, tokens and throttled calls (HTTP 429).
7. Delete deployments you no longer use.

Common errors: `DeploymentNotFound` (wrong name, or the deployment is still being created); `429` (quota); `400` with a content-filter code (the input was blocked, so read the error body; a blocked output instead ends with `finish_reason "content_filter"` in Chat Completions, or `status "incomplete"` with `incomplete_details.reason "content_filter"` in the Responses API); `401` or `403` (sign-in or role problems).

Use it from code

Your first model call. This is Microsoft's documented pattern for the v1 API without keys: you pass a token provider as `api_key`, and the client gets and refreshes tokens for you. Install the `openai` and `azure-identity` packages first.

```python
from openai import OpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider

token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://ai.azure.com/.default")
client = OpenAI(base_url="https://<resource-name>.openai.azure.com/openai/v1/",
                api_key=token_provider)
response = client.responses.create(model="<deployment-name>", input="This is a test")
print(response.model_dump_json(indent=2))
```

Source: Azure OpenAI in Microsoft Foundry Models v1 API

The `model` value is your **deployment name**, not the model's family name. Everything else builds on this client:

- **Chat Completions** still works and is what many frameworks call. Use the Responses API for new code.
- **Structured output** sends a strict JSON Schema and returns data your code can trust. This is the safest way to get data out of text. See How to use structured outputs.
- **Tool calling.** The model never runs your function. It asks for it, your code runs it, and you send the result back. Always cap the number of rounds. See How to use function calling.

- **Embeddings.** Use one embedding model and keep the vector size the same everywhere. See Generate embeddings.
- **Streaming and images as input** (the deployment must be a model that accepts images) are both covered in Use the Azure OpenAI Responses API.
- **Handling errors.** The OpenAI client already retries connection errors and 429 and 5xx responses with growing waits. Raise the retry limit, set a timeout, and catch the typed errors so your app can answer politely. The openai-python repository documents the retry and timeout settings.

Main cost considerations

You are charged for **input tokens** (what you send), **output tokens** (what the model writes, including hidden reasoning tokens for reasoning models), and for any images or audio. Batch jobs get a 50% discount compared with Global Standard.

A simple way to estimate cost without guessing prices: monthly cost ≈ requests × (average input tokens × input rate + average output tokens × output rate) ÷ 1,000,000, using the per-million-token rates on the official pricing page. For example, 200,000 requests at 3,000 input and 400 output tokens each is 600 million input tokens and 80 million output tokens a month. If 2,000 of the 3,000 input tokens are retrieved text, halving the retrieved text in each prompt removes 200 million input tokens.

> **Cost risk — Long prompts and agent loops add up**
> Long instructions and long retrieved documents are sent again with every request. An agent may call the model several times per user question. Measure tokens per *user request*, not per API call.

Common beginner mistakes

- Hard-coding the model name. On Azure, `model=` is your **deployment name**, which is often different from the model name. Keep it in configuration.
- Ignoring the `usage` field. Without it you cannot find out why costs rise.
- Treating 429 as a crash. Retry with a wait, or ask for more quota.
- Putting retrieved documents next to your instructions with no separator. A document can contain text that tries to give the model new orders (**prompt injection**). Wrap documents in tags and tell the model they are data.
- Running model output as code or SQL without checking it first.

Official documentation

- Deployment types for Foundry Models
- Azure OpenAI v1 API and API lifecycle
- Azure OpenAI pricing
- OpenAI Python library (GitHub)

AI Agents and Content Safety

A plain model call answers one question and forgets. An **agent** can take several steps, use tools and remember a conversation. Azure also gives you filters that stop harmful content.

Foundry Agent Service

What it does

An **agent** is a program in which a model decides, step by step, what to do next: call a tool, look something up, ask a question or answer. A **tool** is a function the agent may call, for example "look up an order" or "search the knowledge base". The agent service handles the parts you would otherwise build yourself: conversation state, tool running and tracing.

On Azure the Foundry Agent Service offers:

- **Prompt agents.** You define the model, the instructions and the tools. Foundry runs the agent, so you write no hosting code. You can create one in the portal or with `project.agents.create_version(...)`. Each change is saved as a new **version**.
- **Built-in tools:** file search, code interpreter and web search, plus your own functions, OpenAPI specs and MCP servers (a standard way to expose tools to agents).
- **Conversations.** The service keeps the history, so you only send the new message each time.

The older terms from the Assistants API changed: threads, messages, runs and assistants are now **conversations, items, responses and agent versions**. Foundry also supports **hosted agents**, where you package your own agent code in a container; that is a later topic (see What to Learn Later).

> **Note — Preview features**
> Microsoft's agents overview labels some capabilities as preview (for example voice-based agents and the agent optimizer) and says preview features have no SLA. The overview does not label prompt agents or hosted agents as preview or generally available, so check the current page before relying on a feature in production. In the Python SDK, hosted agent methods became stable in azure-ai-projects 2.3.0 (1 July 2026). Check the overview page for the current status of the service itself.

For a first agent you only need four ideas: the **model deployment**, the **instructions**, one or two **tools**, and a **conversation** (the saved history).

When to use it

Use an agent when a task needs several steps with decisions in between, for example: find the customer, check their order, then decide what to do.

Do not use an agent when a fixed sequence works. A simple pipeline such as *extract the fields*, *check them*, *save them* is cheaper, faster and easier to test as ordinary code with one or two model calls. Many "agents" in real projects are better as a workflow with one model step.

On Azure, use a **prompt agent** for most business agents. Use plain Responses API calls in your own service when you want full control.

Set it up in the portal

Before you start: a Foundry project, a chat model deployment that supports tool calling, and the *Foundry User* role.

1. In the Foundry portal open your project and choose **Agents › New agent**.
2. Choose the model deployment and write the **instructions**: the agent's role, what it may and may not do, and the answer style. Give the agent a clear name. The name and version identify it to your code.
3. Add **tools**. Start with **File search** (upload a few files) or an **OpenAPI** tool. Add more only when you need them.
4. Test in the **agent playground**. Open the **trace** of a run to see each model call, tool call, token count and delay.
5. Save a version. In production your code should call a specific version.
6. Clean up: delete test agent versions, conversations and uploaded files.

Use it from code

Microsoft's Quickstart: Get started with Microsoft Foundry SDK shows how to create a prompt agent with `project.agents.create_version(...)` and a `PromptAgentDefinition` (the model deployment and the instructions), and then talk to it in a conversation. Function calling shows an agent with your own function as a tool: the agent returns a request to call the function, your code runs it, and you send the result back on the same conversation.

Key points:

- **Cap the tool loop.** Allow only a small number of rounds so the agent cannot run forever.
- **Check tool inputs** before you act on them, and ask for confirmation before any action that changes data.
- **Call a specific agent version** in production, so a change in the portal does not change your app without warning.
- **Clean up** test agent versions and conversations when you finish (use a `finally` block or a clean-up script).

Main cost considerations

Agents multiply model calls. One user message may cause several model calls, and each call re-sends the conversation and the tool results. On Azure you pay for model tokens at every step, any built-in tools that have their own meter (such as web search or code interpreter sessions), and storage for files and conversations.

> **Cost risk — Cap every loop**
> Limit the number of tool rounds and the tokens per run. Use a smaller model for simple steps, and test with short conversations first.

Common beginner mistakes

- Letting the loop run without a limit. Always cap the number of tool rounds.
- Giving a tool more power than it needs (for example a "run any SQL" tool). Each tool should do one small thing and check its inputs.
- Trusting tool output blindly. Web pages and documents can contain instructions meant to trick the model.
- Putting secrets in the agent instructions.
- Building an agent when a plain function with one model call would do.

Official documentation

- Foundry Agent Service overview
- Function calling with Foundry agents

Content Safety and guardrails

What it does

Models can be tricked, and users can type harmful things. Safety checks sit on both sides of the model call:

- **On the input:** detect harmful content, jailbreak attempts and **prompt injection** (hidden instructions in a document or web page that try to take control of the model).
- **On the output:** detect harmful content, protected material and answers that are not supported by your sources.

On Azure, **content filters** (also called guardrails) come with a default configuration on Foundry model deployments. You can create your own configuration and change the strictness for categories such as hate, sexual content, violence and self-harm. **Azure AI Content Safety** is also a separate API you can call on any text or image, including the output of other models. It includes **Prompt Shields**, which look for prompt-injection attacks.

When to use it

Every feature that shows model output to a user needs at least the default filters. Add your own rules where the business needs them, for example blocking certain topics or hiding personal data.

Filters reduce risk but do not remove it, and they can block legitimate content if they are too strict. They are not access control. Never rely on a filter to stop someone seeing data they are not allowed to see. Enforce that in your search and database queries.

Set it up in the portal

1. In the Foundry portal open **Guardrails** (content filters) and choose **Create**. Set the strictness for input and output per category, and turn on **Prompt Shields** for jailbreak and indirect attacks.
2. Assign the configuration to your model deployment (and agents). Test it with a few known attack phrases and with some normal questions to check it does not block good content.
3. To call the separate API, create a **Content Safety** resource (or use the Content Safety capability of your Foundry resource) and try the tools on the Content Safety try-out pages.
4. Remember the symptoms: a blocked input returns HTTP **400** with a content-filter error code. A blocked output shows up differently by API: in Chat Completions, `finish_reason` is `"content_filter"`; in the Responses API, the response `status` is `"incomplete"` with `incomplete_details.reason` set to `"content_filter"`.

Use it from code

The Quickstart: Analyze text content shows how to check a piece of text with the `azure-ai-contentsafety` package. The result lists each category (hate, sexual, violence, self-harm) with a severity. By default the service returns four severity levels: **0, 2, 4 or 6** (safe, low, medium, high). Decide which level your app blocks.

The Quickstart: Prompt Shields shows the REST call that checks a user prompt for direct attacks and a list of documents for hidden instructions. The quickstart uses API version `2024-09-01`. The response says, for the prompt and for each document, whether an attack was detected.

Key points:

- Call Prompt Shields on retrieved documents **before** you send them to the model.
- If an attack is detected, block the request or send it for review. Do not just retry.
- Sign in with Entra ID rather than a key where the service supports it.

Main cost considerations

On Azure, default content filtering on Foundry model deployments is included with model usage. The separate Content Safety API is billed per text record and per image. Every extra check adds delay, so turn on only what the feature needs.

Common beginner mistakes

- Shipping a chat feature with no filter or injection check.
- Using a very strict filter, then blocking normal questions. Test with real examples.
- Treating a filter as a permission system.
- Putting retrieved documents straight into the prompt without checking or fencing them.

Official documentation

- Azure AI Content Safety overview
- Prompt Shields concepts
- Quickstart: Prompt Shields (REST)
- Azure AI Content Safety Python SDK (PyPI)

OCR and Document Extraction

Many real AI projects begin with documents: PDFs, scans, forms, invoices and receipts. Models cannot read a scanned page as text. **OCR** (optical character recognition) turns images of text into real text, and modern services also keep tables and layout.

Document Intelligence

What it does

Azure Document Intelligence reads documents and returns structured results. It is part of Foundry Tools. It offers:

- **Layout model** (`prebuilt-layout`): text, paragraphs, tables, checkboxes and headings. It can return **Markdown**, which is ideal input for splitting a document into chunks for RAG.
- **Read model:** plain OCR.
- **Prebuilt models** for common documents such as invoices, receipts and ID documents. They return named fields (vendor, total, date) with a **confidence** score for each.
- **Custom models** that you train on your own labelled documents.

Analysis is a **long-running operation**: you submit the document, then wait for the result. The Python SDK hides this with a *poller*. The package is `azure-ai-documentintelligence`.

When to use it

- **Use Document Intelligence** for scanned or image-only PDFs. Without OCR there is no text to work with.
- **Use the layout model** for documents with tables, forms or checkboxes. It keeps their structure, and its Markdown output keeps headings and tables intact when you split documents for RAG.
- **Use a prebuilt model** for invoices, receipts and ID documents. It returns named fields with a confidence score for each.
- **Consider sending the page image to a multimodal model instead** when you have only a few documents or are running a quick experiment. You lose page layout and confidence scores, and the cost grows with image size.

Do not feed scanned PDFs straight into a text splitter: with no OCR the chunks are empty. Do not flatten tables into plain text either. Use the Markdown output so tables keep their meaning. For important fields (amounts, dates), check the confidence score and send low-confidence values to a person for review.

Set it up in the portal

1. **Get a resource.** Use your Foundry resource (Document Intelligence is available through it), or create a **Document Intelligence** resource in a supported region. Use a custom subdomain name, because it is needed for sign-in with Entra ID.
2. **Try it first.** Open **Document Intelligence Studio**, upload a sample PDF and run the **Layout** and **Invoice** models. Look at the Markdown output, the tables and the confidence scores.
3. **Give your code access.** In **Access control (IAM)** assign **Cognitive Services User** (standalone resource) or **Foundry User** (Foundry resource) to the user or identity that runs your code.
4. **Test from code.** Common errors: `InvalidRequest` (unsupported file type or size), `401` or `403` (sign-in or role) and time-outs on long documents.
5. **Clean up.** Delete custom models and test storage you no longer need.

Use it from code

The Azure AI Document Intelligence client library for Python shows how to analyse a local file or a URL with `begin_analyze_document` and a model ID such as `prebuilt-layout` or `prebuilt-invoice`, and how to sign in with `DefaultAzureCredential`.

Key points:

- Analysis is a long-running operation: the SDK returns a *poller*, and you wait for its result.
- For RAG, ask the layout model for Markdown output (see the SDK reference for the option), so headings and tables survive chunking.
- For invoices and other prebuilt models, read each field's **confidence**. Send values below your threshold (for example 0.8), and any missing field, to a person for review.
- Catch service errors and record which file failed, so one bad file does not stop a batch.

Main cost considerations

Document Intelligence charges **per page**. Read costs least; layout and prebuilt models share a higher rate; custom extraction costs the most. Save money by:

- not re-processing a file that has not changed (store a hash of each file and skip repeats);
- analysing only the pages you need;
- choosing the simplest model that gives the fields you need.

The service also limits how many requests you can send per second, so process large batches gradually.

Common beginner mistakes

- Sending scanned PDFs to a text splitter with no OCR step.
- Throwing away tables by using plain text instead of Markdown.
- Trusting extracted numbers without checking the confidence score.
- Re-processing the same document every time the pipeline runs.
- Leaving results in the service: async results are kept only for a limited time, so save them.

Official documentation

- Azure Document Intelligence overview
- azure-ai-documentintelligence (PyPI)

Search and RAG

A model only knows what it was trained on. **Retrieval-augmented generation (RAG)** lets it answer from *your* documents. The idea is simple: find the passages most relevant to the question, put them in the prompt, and tell the model to answer only from them and to say which passage it used. In practice, the quality of the search matters more than the choice of model. This chapter covers the search service, the RAG pipeline and how to test it.

Azure AI Search

What it does

Azure AI Search is a managed search engine. You put your content in an **index**, then query it by keywords, by meaning, or both.

- **Index.** A table-like structure with fields. Text fields can be searched. Number and category fields can be *filtered*. A **vector field** stores an embedding (a list of numbers describing meaning).
- **Queries.** *Keyword* search matches words. *Vector* search matches meaning. **Hybrid** search does both and merges the results, and it usually beats either one alone. A filter narrows results (for example only one department).
- **Semantic ranker.** An optional extra step that re-orders the top results to put the best ones first.
- **Indexers and skillsets.** An **indexer** pulls files from a source such as Blob Storage on a schedule. A **skillset** adds steps while indexing: reading text, splitting it into chunks and creating embeddings. This is called **integrated vectorization**. Nothing happens automatically: chunking and embedding run only if the skillset contains a chunking skill and an embedding skill, and the index has matching vector fields. The portal wizard creates these objects for you from your choices, but they are still separate objects (data source, skillset, index, indexer) that you can open and edit.
- **Tiers.** Free and Basic suit learning and small projects. Standard tiers add capacity. On dedicated tiers, **replicas** add query capacity and **partitions** add storage.

When to use it

Use AI Search for most RAG apps on Azure. Hybrid search with the semantic ranker works well on business documents, and the import wizard saves a lot of code.

Consider something simpler if you have only a small amount of data that already lives in PostgreSQL (see PostgreSQL and pgvector), or if you are building a throw-away prototype and do not want to pay for a search tier.

Set it up in the portal

Before you start: a Foundry resource with an *embedding* model deployed (and a chat model), a storage account with a few documents, and permission to create resources.

1. **Create the service.** Azure Portal › **Create a resource › Azure AI Search**. Pick the subscription, resource group, a unique name, the region (the same region as Foundry and Storage if possible) and a pricing tier. Use **Basic** while you learn: the Import data quickstart says Basic or higher is needed for managed identity.
2. **Turn on role-based access.** Under **Settings › Keys** choose **Role-based access control** (or *Both*). Then in **Access control (IAM)** give your user **Search Service Contributor** and **Search Index Data Contributor**. Later your application's identity only needs **Search Index Data Reader**.
3. **Let the service read your files and call the model.** Turn on the service's system-assigned managed identity. Give it **Storage Blob Data Reader** on your storage account. For an Azure OpenAI embedding deployment, Microsoft's integrated-vectorization article says to give the same identity **Cognitive Services OpenAI User** on that resource. For a Foundry resource, check the current article for the role (not confirmed here). Also check the quickstart's prerequisites: public network access must be on while the wizard runs, and the Azure OpenAI option needs a

custom subdomain.

4. **Import and vectorize.** On the service's Overview page choose **Import data**, select Blob Storage as the data source, then **RAG**. Pick your embedding model. In the quickstart's wizard, chunking uses fixed settings (pages of 2,000 characters with 500 characters of overlap), so you cannot tune chunk size there; to change it you edit the skillset afterwards or build the pipeline in code. At the end the wizard creates the data source, skillset, index and indexer, and you can open each one under **Search management**. Set the schedule to **Once** while learning.
5. **Test.** In **Search explorer** run a keyword query, a vector query and a hybrid query. Turn on the semantic ranker and try a filter.
6. **Fix problems.** The indexer's run history lists errors for each document (unsupported file, time-out, throttling by the embedding model). `403` means a missing role. `404` means the wrong index name.
7. **Clean up.** Delete indexers first (they run on a schedule and call paid services), then indexes, then the service.

Use it from code

The Azure AI Search client library for Python shows how to create an index with `SearchIndexClient`, upload documents and run a query. Create a vector query shows vector and hybrid queries.

Key points:

- The vector field's **dimensions must match your embedding model**. Changing the model means a new index.
- Give each chunk a **stable ID** and upload with *merge or upload*, so running the job twice does not create duplicates.
- Add a **vectorizer** to the index, so the service can create the query embedding for you.
- For the best results, combine keyword search, vector search and the semantic ranker in one query.
- Mark fields you want to filter on as *filterable*, and return the source fields (title, link) you need for citations.

Main cost considerations

On Azure the main charge is for **search units** (replicas × partitions), billed by the hour on dedicated tiers *whether or not anyone searches*. The semantic ranker has a free monthly allowance and then per-request charges. Indexing with skillsets also uses model tokens and Foundry Tools transactions.

> **Cost risk — Re-indexing repeats the embedding bill**
> Every full re-index re-creates every embedding. Only re-process files that changed, and keep embeddings you have already made, keyed by a hash of the chunk text and the model name.

> **Cost risk — Search services bill while idle**
> A dedicated Azure AI Search tier is charged by the hour. Delete test services when you finish.

Common beginner mistakes

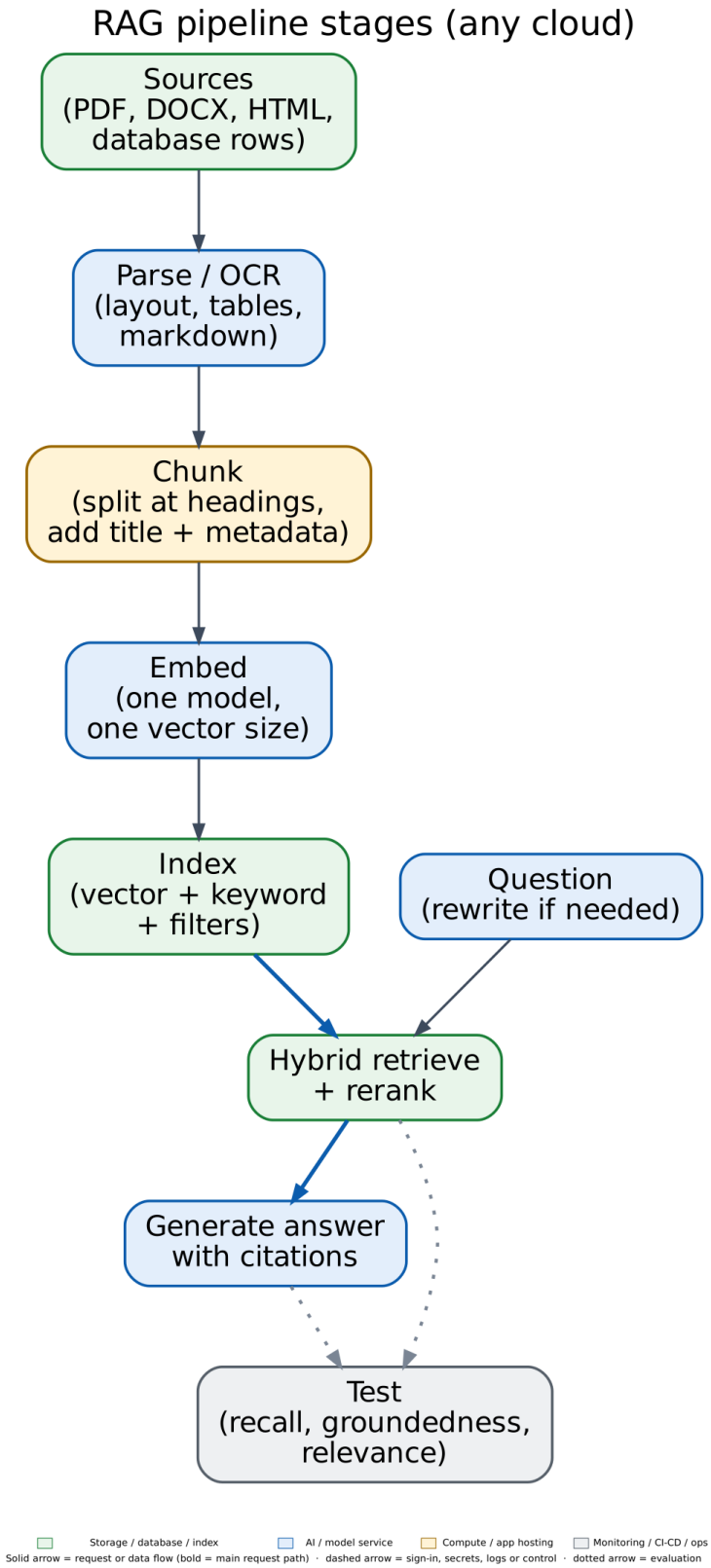
- **Vector-only search.** It can miss exact matches such as product codes and error numbers. Use hybrid search.
- **Different embedding models or sizes** at index time and query time. They must match exactly.
- **Chunks that are too big or too small.** Big chunks dilute relevance and small ones lose context.
- Giving yourself *Owner* but not a **Search Index Data** role, then getting `403`.
- Forgetting to make a field `filterable` before you need it. Many index changes require re-indexing.

Official documentation

- What is Azure AI Search?
- Vector search overview
- Hybrid search overview
- Semantic ranking
- Azure AI Search / Foundry IQ pricing
- azure-search-documents (PyPI)

RAG basics: from documents to answers

What it does



The RAG pipeline: offline ingestion (parse, chunk, embed, index) and online retrieval (rewrite, hybrid retrieve and rerank, generate with citations), with evaluation across both.

A RAG app has two halves.

Ingestion (done ahead of time):

1. **Collect** the documents and keep useful details with each one: its source link, title and date.

2. **Parse or OCR** them into clean text that keeps headings and tables. Use Document Intelligence for scans and complex PDFs.
3. **Chunk** the text. A **chunk** is a passage small enough to be useful on its own. Good defaults: split at headings and keep tables whole; for plain text use chunks of roughly 300 to 800 tokens with 10 to 20 percent overlap; put the document title and section name at the top of each chunk.
4. **Embed** each chunk with one fixed model. Always use the same model and vector size for the whole index.
5. **Index** the text, the vector and the details in your search service. Give each chunk a stable ID. Before you re-index a changed document, delete its old chunks, or stale chunks will still be retrieved.

Answering (done for each question):

1. **Retrieve.** Search with keywords and meaning together (hybrid), apply any filter, and keep the best 3 to 8 chunks.
2. **Build the prompt.** Put the instructions first (answer only from the sources, cite them, say "I don't know" if the answer is missing), then the chunks inside clear tags, then the question.
3. **Generate.** Ask the model for a structured answer that includes the IDs of the chunks it used.
4. **Check and return.** Confirm that every cited ID really was one of the chunks you sent, then return the answer with links to the sources.

To reduce made-up answers (hallucinations): improve retrieval first, allow "I don't know", require citations and check them, use a low temperature, and keep chunks focused.

When to use it

Use RAG when answers depend on private, changing or citable information. Use fine-tuning (a later topic) to change *behaviour or format*, not to teach facts. For a very small set of documents, you can sometimes put whole documents in the prompt, if the cost and delay are acceptable.

Set it up in the portal

The quickest start is the **Import data** wizard in Azure AI Search (see Azure AI Search), then the Foundry playground's option to add your data, or an agent with a file search or Azure AI Search tool. Use this to get a first working baseline, then move to code so you control chunking and prompts.

Use it from code

Two pieces of code make up most RAG apps. Microsoft's Design and develop a RAG solution guide covers both in depth, and Chunk large documents covers chunking.

- **The chunker.** Split Markdown at headings, keep tables whole, put the document title and section name at the top of each chunk, and build each chunk's ID from the document ID and its position, so the ID stays the same every time you re-index. This works on any cloud because it is plain Python.
- **The answer step.** Retrieve the best chunks (see Azure AI Search), put each one in the prompt inside tags with its ID, tell the model to answer only from them and to cite the IDs, and ask for structured output. Then **check the citation IDs**: drop or flag any cited ID that was not among the chunks you sent. Always allow "I don't know".

Main cost considerations

The cost of one answer is roughly the query embedding, plus the search query, plus the generation tokens (instructions + retrieved chunks + question + answer). The retrieved chunks are usually the biggest part: five 600-token chunks add 3,000 input tokens to every question. Tune how many chunks you send and how big they are, cache answers to repeated questions, and use a small model for simple steps. Ingestion costs parsing pages, embedding tokens and indexing; only process changed files.

Common beginner mistakes

- Blaming the model when the real problem is retrieval. Look at what was retrieved first.
- Not allowing "I don't know", which pushes the model to invent an answer.
- Not checking that the cited IDs were really in the retrieved set.
- Putting retrieved text in the prompt without tags, which lets a document give orders to the model.
- Ignoring document permissions. If different users may see different documents, filtering results by permission is essential. This topic is covered under What to Learn Later; do not put private documents in a shared index until you have learned it.

Official documentation

- RAG solution design and evaluation guide (Azure Architecture Center)
- Chunking large documents for vector search

Testing your answers

What it does

AI apps fail quietly: a wrong answer looks just like a right one. **Evaluation** turns quality into numbers you can track, the way unit tests do for ordinary code. Start with two simple checks:

- **Did search find the right documents?** The measure is **recall@k**: for each test question, was a correct document among the top *k* results?
- **Is the answer supported and relevant?** On Azure, the open-source `azure-ai-evaluation` library has *evaluators* such as `GroundednessEvaluator` (is the answer backed by the sources?) and `RelevanceEvaluator` (does it answer the question?). These use a "judge" model to score each answer.

When to use it

Build a **golden set** of 30 to 100 real questions, with the document or facts that should answer each one, and include a few questions the documents cannot answer. Run it every time you change the prompt, the model, the chunk size or the search settings. A judge model is cheaper than people but not perfect, so spot-check some scores by hand.

Set it up in the portal

1. In the Foundry portal open your project and choose **Evaluation › Create**. Pick what to test: a model deployment, an agent, or a dataset of answers you already have.
2. Upload a JSONL dataset with fields such as `query`, `context`, `response` and `ground_truth`.
3. Choose evaluators (quality is enough to start) and the judge model deployment.
4. Run it and read the scores and the reasons for each row.

Use it from code

The azure-ai-evaluation package provides evaluators such as groundedness and relevance, which use a model deployment as the judge. Microsoft's Observability in generative AI explains how evaluations fit with tracing and monitoring.

Key points:

- Keep the judge model's settings outside your code (configuration or Key Vault).
- **Measure search quality with recall@k** before you judge answers. For each golden question, check whether a correct document ID appears in the top *k* results; recall@k is the share of questions where it does. This is plain Python if your retrieval returns documents with a document ID.

- Run the same golden set after every change to prompts, chunking or models, and compare the scores.

Main cost considerations

Each AI-assisted score is a call to the judge model. Five hundred questions × four metrics is 2,000 model calls for one run. Use a small golden set for everyday checks and the full set less often. Managed evaluation jobs have their own charges.

Common beginner mistakes

- Measuring only the final answer. If retrieval is wrong, the answer cannot be right, so measure retrieval separately.
- Changing several things at once, so you cannot tell what helped.
- Testing on the same questions you used to write the prompt.
- Not saving the scores together with the version of the prompt and settings.

Official documentation

- Observability in Microsoft Foundry
- azure-ai-evaluation package (PyPI)

Storage and Databases

AI apps need places to keep things: uploaded documents, chat history, results and embeddings. This chapter covers the three storage services a junior AI Engineer uses most: object storage for files, PostgreSQL for application data and vectors, and Cosmos DB for chat history.

Blob Storage

What it does

Azure Blob Storage stores files (called **blobs**) in **containers** inside a **storage account**. It is the usual place for the documents you will search, the PDFs you will OCR and the files your app produces. Blobs have **access tiers** (Hot, Cool, Cold, Archive): colder tiers generally cost less to store but more to read. **Lifecycle rules** can move or delete blobs automatically.

Access tiers apply to block blobs. The first three tiers are *online*: the data is readable in milliseconds. **Archive** is *offline*: you cannot read it until you move it back to an online tier (this is called **rehydration**).

Tier	Online or offline	Minimum time to keep a blob*	Typical use
Hot	Online	None	Files you read often
Cool	Online	30 days	Files you read rarely
Cold	Online	90 days	Files you read very rarely but may still need quickly
Archive	Offline	180 days	Files you almost never need and can wait hours for

*For general-purpose v2 accounts. Microsoft's documentation says Blob Storage (legacy) accounts have no minimum for Cool or Cold. If you delete, overwrite or move a blob before its minimum time, you pay an **early-deletion charge** for the remaining days. The account default tier can be Hot, Cool or Cold (not Archive); you can set a different tier on each blob.

Archive rehydration. An archived blob cannot be read or changed. To use it, move it to Hot, Cool or Cold. According to Microsoft this can take up to 15 hours, depending on the priority you choose (*standard* or *high*, which differ in speed and cost). While it is rehydrating, you are still billed for it as archived data. Lifecycle management rules cannot rehydrate blobs.

For a first RAG project keep your source documents in **Hot** or **Cool**. Use Archive only for files you will not search.

Idea	Name
Top-level container	Storage account
Group of files	Container
A file	Blob
Sign-in for code	Entra ID and *Storage Blob Data* roles
Temporary download link	SAS (shared access signature)

When to use it

Use Blob Storage as the first stop for almost any file: source documents for RAG, scanned PDFs and images waiting for OCR, and the results your pipelines produce. It is also the usual data source for an Azure AI Search indexer (see Azure AI Search).

Do not use it as a database. Do not store millions of tiny files if you can avoid it, because each request has a cost.

Set it up in the portal

1. **Create the account. Create a resource › Storage account.** Choose a name (3 to 24 lowercase letters or digits, unique worldwide), a region and *Standard* performance. For learning, locally redundant storage (LRS) is enough.
2. **Check the safety settings** on the *Advanced* tab: leave *Allow Blob anonymous access* **off**, and turn **off** *Allow storage account key access* so that only Entra ID sign-in works. Use TLS 1.2 or higher.
3. **Give yourself access.** In **Access control (IAM)** add the role **Storage Blob Data Contributor** for your user. *Owner* alone does not let you read or write the blob data when you sign in with Entra ID.
4. **Create a container** (for example `documents`) and upload a file in the **Storage browser**.
5. **Optional: save money.** Under **Lifecycle management** add a rule such as "delete files in `tmp/` after 7 days".
6. **Fix problems.** `403 AuthorizationPermissionMismatch` means you lack a data role. `AuthorizationFailure` means a network rule is blocking you.
7. **Clean up.** Delete the storage account. Deleting blobs before their tier's minimum time (see the table above) can add an early-deletion charge.

Use it from code

Install the `azure-storage-blob` and `azure-identity` packages. The Azure Storage Blobs client library for Python shows how to create a `BlobServiceClient` from the account URL and `DefaultAzureCredential`, and how to upload, list and download blobs.

Key points:

- "Folders" are just name prefixes such as `hr/policy.pdf`. List blobs by prefix to see one folder.
- Say whether an upload may overwrite an existing blob; by default it fails if the blob exists.
- Stream large files instead of reading them all into memory.

A temporary download link. Share a file for a short time without giving anyone access to the whole account. A *user-delegation SAS* is signed with your Entra sign-in rather than the account key. The signing identity needs a role that can get a user delegation key (for example *Storage Blob Delegator* or a Storage Blob Data role at account level). Microsoft documents that such a SAS can be revoked by revoking the key or changing the signer's role assignments, with a possible delay. Keep it short-lived. See Create a user delegation SAS.

Main cost considerations

You pay for **storage** (per GB per month, depending on the tier and redundancy), **operations** (writes, reads and list calls, which add up with many small files), **data retrieval** from cold tiers, and **data leaving** the region. Keep your compute in the same region as your data, use lifecycle rules, and avoid the Archive tier for files you read often.

Common beginner mistakes

- Making a container public "just for testing" and forgetting about it.
- Putting the storage account key in code. Sign in with Entra ID instead.
- Being *Owner* but not having a **Storage Blob Data** role, then getting `403`.
- Leaving test data behind with no lifecycle rule.

Official documentation

- Introduction to Azure Blob Storage
- Access tiers for blob data
- Azure Storage Blob client library for Python

PostgreSQL and pgvector

What it does

Azure Database for PostgreSQL – Flexible Server is the PostgreSQL database run for you. It handles backups, updates and sign-in. It supports the `vector` extension (**pgvector**), which lets you store embeddings in a column and find the closest ones with a normal SQL query. That makes PostgreSQL a simple choice for **vector search** when you have a small or medium amount of data.

When to use it

Use PostgreSQL for structured application data: users, document catalogues, ingestion status, evaluation results. Add pgvector when you want vectors next to that data and your collection is small to medium, so you do not need a separate search service.

Choose Azure AI Search instead for large document collections, hybrid keyword-and-meaning ranking, and when you want a ready-made indexing pipeline (which you still configure, or let the wizard configure).

Set it up in the portal

1. **Create the server. Create › Azure Database for PostgreSQL flexible server.** Pick the *Development* workload preset, the **Burstable** compute tier for learning, and your region.
2. **Authentication.** Choose **PostgreSQL and Microsoft Entra authentication** (or *Microsoft Entra authentication only*) and set yourself as the Entra admin.
3. **Networking.** For learning, choose public access with a firewall rule that allows only your own IP address. Never allow the whole internet.
4. **Allow the extension.** Open **Server parameters**, find `azure.extensions`, add **VECTOR** and save.
5. **Create the extension** from any SQL client: `CREATE EXTENSION vector;`
6. **Fix problems.** `no pg_hba.conf entry` points to a network or firewall rule. `password authentication failed` usually means an expired token or an unmapped role.
7. **Clean up.** Delete the server, or stop it when you are not using it. A stopped server starts again automatically after 7 days and is billed again, so delete it if you are done.

Use it from code

Microsoft's Enable and use pgvector shows how to enable the extension, create a table with a `vector` column and find the closest rows with SQL. The pgvector project documents the index types and distance operators. From Python you connect with the `psycopg` driver and register the `pgvector` type.

Key points:

- Sign in with an Entra token instead of a stored password: get a token with `DefaultAzureCredential` and pass it as the password.
- The size of the `vector` column must equal your embedding model's vector size.
- `<=>` means "cosine distance", so a smaller number means a closer match. Always use `LIMIT`.
- Load your data first and create the index afterwards if you load a lot of rows, because building an index while loading is slow.
- Always pass values as query parameters, never by joining strings.

Main cost considerations

On Azure you pay for **compute** (the number of vCores), **storage and input/output**, and **backup storage** beyond the free allowance. High-availability standby servers roughly double the compute cost. For vector search, memory matters: HNSW indexes work best when they fit in memory.

Common beginner mistakes

- Opening the database to the whole internet to avoid dealing with a firewall rule.
- Storing the database password in code. Use an Entra token, or Key Vault.
- Forgetting `LIMIT` in a vector query.
- Using a different vector size in the table than your embedding model produces.
- Never testing a restore from backup.

Official documentation

- Azure Database for PostgreSQL flexible server overview
- Use pgvector on Azure Database for PostgreSQL

Cosmos DB

What it does

Azure Cosmos DB is a managed NoSQL database that stores JSON documents. For AI apps you will usually use the **NoSQL API**. The hierarchy is **account → database → container → item**: an *item* is one JSON document, and a *container* is where items are stored. Every container has a **partition key**, a field (for example `conversationId`) that decides how the data is spread out. Queries that include the partition key are fast and cheap. Throughput is measured in **Request Units (RU/s)**. You can pay for provisioned RU/s or choose **serverless**, which bills only for what you use and suits learning.

Cosmos DB has other APIs too (for example MongoDB, Cassandra, Gremlin and Table). They name things differently: a NoSQL *container* is a *collection* in the MongoDB API and a *table* in the Cassandra and Table APIs. This guide covers only the NoSQL API. Cosmos DB for NoSQL does support vector search, but for a first project PostgreSQL with pgvector is simpler. See the Cosmos DB vector search link in the documentation list below if you want to explore it later.

When to use it

Cosmos DB is a good home for **chat history**, agent state, user feedback and other JSON data that you read by key (for example "the last 20 messages of this conversation"). Choose a partition key that matches your most common query, such as `/conversationId`.

Set it up in the portal

1. **Create a resource › Azure Cosmos DB › Azure Cosmos DB for NoSQL.** Choose **Serverless** capacity mode for learning, and your region.
2. **Give your code access.** Cosmos DB has its own data-plane roles that are separate from normal Azure roles. Assign **Cosmos DB Built-in Data Contributor** (role definition ID `00000000-0000-0000-0000-000000000002`, as shown in Microsoft's article) to your user with `az cosmosdb sql role assignment create`, giving the account, the role definition ID, your principal ID and scope `/` for the whole account. Azure roles such as *Cosmos DB Account Reader* do not give access to the data.
3. **Create a database and container** in **Data Explorer**. Set the partition key (for example `/conversationId`) and turn on **Time to live (TTL)** so old messages are deleted automatically. If you later turn off key-based access,

create databases and containers with the CLI or infrastructure code instead.

4. **Fix problems.** `403` with "request blocked by Auth" means a missing data role. `429` means you used up the RU/s; the SDK retries automatically, but you may need to optimise queries or raise throughput.
5. **Clean up.** Delete the account.

Use it from code

Install the `azure-cosmos` and `azure-identity` packages. The Azure Cosmos DB client library for Python shows how to create a `CosmosClient` with the account endpoint and `DefaultAzureCredential`, and how to write, read and query items.

Key points:

- Every item needs an `id` and a value for the container's **partition key** (for example `conversationId`).
- **Use the partition key** in reads and queries. A query that names it reads one logical partition, which is fast and cheap.
- A per-item `ttl` value only works if Time to live is turned on for the container.
- Use parameterised queries, never string joins.
- On `429`, the SDK retries for you; if it keeps happening, optimise the query or raise the throughput.

Main cost considerations

Cosmos DB charges for **Request Units** (provisioned or, in serverless, consumed), **storage**, and any extra regions you add (each region multiplies the cost). Reading one item by its ID and partition key is the cheapest operation; queries and vector searches cost more. Serverless or autoscale keeps idle development accounts cheap.

Common beginner mistakes

- Choosing a partition key with few distinct values, which creates a "hot" partition.
- Running queries across all partitions in the main request path.
- Letting items grow forever. Use TTL.
- Using the account key in code instead of Entra ID and a data role.
- Leaving an over-provisioned account running for weeks.

Official documentation

- Azure Cosmos DB introduction
- Azure Cosmos DB resource model
- Azure Cosmos DB Python SDK
- Vector search in Azure Cosmos DB for NoSQL

Running Your Python App

A notebook that calls a model is not yet an application. To let other people use your work, you must run your code somewhere that stays online, starts when needed and can be updated safely. This chapter covers the simplest managed options: **containers** for web APIs and **functions** for small event-driven tasks. You will not manage any servers.

Azure Container Apps

What it does

A **container** packages your code and everything it needs to run, so it behaves the same on your laptop and in the cloud. **Azure Container Apps** runs containers for you. You give it an image and it provides a web address, HTTPS, scaling and logs.

The words to know:

- **Environment.** The shared home for your container apps, with a shared place for logs.
- **Container app.** Your running container with its settings: CPU and memory, environment variables and **ingress** (the setting that gives it a web address).
- **Revision.** An unchangeable version of your app. Each update creates a new revision, so you can roll back.
- **Scaling.** The app can run more copies when traffic grows, and with the Consumption plan it can scale down to zero copies when nobody is using it.

When to use it

Container Apps is a good first home for a Python API that calls models: a RAG API, an agent backend or a background worker. Use Azure Functions instead for tiny event handlers. Move to Kubernetes only if you have needs that justify it (see What to Learn Later).

Set it up in the portal

You need a container image in a registry first. See Azure Container Registry below.

1. **Create a user-assigned managed identity** (**Create › Managed Identity**, for example `id-ragapi-dev`). This is your app's own sign-in. See Managed identities.
2. **Give the identity access.** Add **AcrPull** on the container registry, **Foundry User** on your Foundry resource, and any other roles your app needs, for example **Search Index Data Reader**.
3. **Create the app. Create › Container App.** Create a new **environment** (the Consumption plan is enough). Under *Container*, choose your image from the registry and sign in to the registry with the managed identity. Set CPU and memory (the smallest options are fine).
4. **Turn on ingress.** Choose *Accepting traffic from anywhere* for a public API you want to try, and set the **target port** to the port your app listens on (for example 8000).
5. **Choose scaling.** Set a minimum and maximum number of replicas. A minimum of 0 saves money but causes a slow first request after idle time. Use a minimum of 1 for anything people wait on.
6. **Add settings.** Add environment variables such as `AZURE_OPENAI_BASE_URL`, `CHAT_DEPLOYMENT` and `AZURE_CLIENT_ID` (the managed identity's client ID). Do not put secrets in plain variables; see Key Vault.
7. **Check it.** Open the app URL. Use **Log stream** and **Console** to see what is happening. Common problems: the image will not pull (missing *AcrPull* or identity), health checks fail (wrong port or slow start), `503` while the app starts from zero.
8. **Clean up.** Delete the app, the environment and the Log Analytics workspace (log storage is billed).

Use it from code

Microsoft's Quickstart: Build and deploy your app from your local file system to Container Apps shows how to turn a folder of code into a running container app with the Azure CLI.

Key points for an AI API:

- Add a simple **health endpoint**, and make the app listen on the port you set as the *target port*.
- Create the model client **once at start-up** and reuse it for every request.
- Sign in with `ManagedIdentityCredential` in Azure (pass the identity's client ID, which you set in `AZURE_CLIENT_ID`) and with `DefaultAzureCredential` on your laptop.
- Keep the image small (a slim Python base image) and run it as a non-root user.
- Give every image a **version tag** and update the app to the new tag; avoid `latest`.

Main cost considerations

What you pay depends on the **plan** your app uses. Both plans can exist in one environment.

- **Consumption plan (serverless).** Microsoft's billing page says you pay for the resources your running replicas use (vCPU-seconds and GiB-seconds, billed per second) and for HTTP requests that come from outside the environment. A revision scaled to zero replicas has no resource-consumption charge. A lower *idle* rate applies only in a narrow case: the revision has a minimum replica count above zero and the replica is truly idle. The page also describes a monthly free grant per subscription; check it for the current amounts.
- **Dedicated plan (workload profiles).** You pay for the workload-profile instances, not for each app, plus a plan-management fee when you use dedicated profiles.
- **Other resources.** Log Analytics, a registry, your own virtual network or private endpoints can add cost.

This guide gives no prices. Use the Container Apps pricing page and the Azure pricing calculator for your region.

> **Cost risk — Remember the log workspace**
> The Log Analytics workspace that stores your logs is billed by the amount of data. Delete it when you remove test environments.

Common beginner mistakes

- Scale to zero on an API that people wait on (slow first request). Use at least one replica.
- Wrong **target port**, so health checks fail.
- Pulling the image with the registry's admin password. Use a managed identity with **AcrPull**.
- Putting secrets in plain environment variables.
- Using the tag `latest` for production images. Use explicit version tags.

Official documentation

- Azure Container Apps overview

Azure Container Registry

What it does

A **container registry** is a private store for your container images. Azure Container Registry (ACR) comes in Basic, Standard and Premium tiers. **ACR Tasks** can build your image in the cloud with one command, so you do not need Docker on your own computer.

When to use it

Every container you deploy needs a registry close to where it runs. Use a private registry, not a public one, for your own code.

Set it up in the portal

Create › Container Registry. Choose a unique name, a region and the **Basic** tier for learning. Leave the **admin user disabled**. Give your app's identity the **AcrPull** role. Give the person or pipeline that pushes images the **AcrPush** role.

Use it from code

Sign in with `az login` first. Build a container image on demand in Azure shows how to create a registry and build an image in the cloud with `az acr build`, so you do not need Docker on your laptop.

Key points:

- Use the **Basic** tier for learning and keep the **admin user disabled**.
- Tag every image with a version (for example `1.0.0`), not `latest`.
- Give your app's managed identity the **AcrPull** role on the registry. Assign Azure roles using Azure CLI shows how.

Main cost considerations

ACR charges a daily fee for the tier plus storage above the included amount and build time. Large images (for example, those with GPU libraries) cost more to store and are slow to pull, so keep images small.

Common beginner mistakes

- Turning on the admin user and sharing its password.
- Using `latest` tags in production.
- Never deleting old images, so the registry fills up.

Official documentation

- Introduction to Azure Container Registry

Azure Functions

What it does

Azure Functions runs a small piece of code when something happens. The thing that starts it is a **trigger**: an HTTP request, a timer, a new file in Blob Storage, a message in a queue. You pay only when the code runs. The recommended serverless plan is **Flex Consumption**. The Python **v2 programming model** uses decorators to declare triggers.

When to use it

Azure Functions is ideal for **glue code**: react to a file uploaded to Blob Storage, run something on a timer, or handle a queue message. It also suits a small HTTP endpoint, a small tool for an agent, or one step in an OCR pipeline, such as sending each new file for analysis.

Do not use it for long-running jobs with many heavy libraries or for anything that needs a GPU. Use a container for those. Container Apps is better for a full API.

Set it up in the portal

1. **Create › Function App.** Choose the **Flex Consumption** hosting option and the **Python** runtime. Choose a storage account. Keep **Application Insights** turned on so you can see logs and errors.
2. **Turn on a managed identity** for the function app. Give it the roles it needs on the services it uses, for example **Storage Blob Data Owner** for a blob trigger.
3. **Add settings** under **Configuration**. Use identity-based connections (`<CONNECTION>__blobServiceUri` and `<CONNECTION>__credential=managedidentity`) instead of storage keys.
4. **Use the Event Grid blob trigger.** On Flex Consumption, Microsoft's blob trigger reference says only the event-based Blob storage trigger is supported, not the default polling trigger. Create an Event Grid subscription on the blob container (or use the **Azure Blob Storage trigger (using Event Grid)** template), and set the trigger's source to EventGrid (`source="EventGrid"` in the Python v2 decorator).
5. **Deploy** with `func azure functionapp publish <app-name>`, from VS Code, or from CI/CD. Test with **Code + Test** or an HTTP call.
6. **Fix problems.** In Application Insights open **Failures** and **Live Metrics**. A trigger that never fires usually means a missing role or the wrong connection name.
7. **Clean up.** Delete the function app, its plan and its storage account.

Use it from code

The Azure Blob storage trigger for Azure Functions page shows a Python function that runs when a new file arrives, using the Python v2 programming model (functions defined with decorators in `function_app.py`).

Key points:

- Use **identity-based connections** (the `<CONNECTION>__*` settings) instead of storage keys.
- **On Flex Consumption, use the Event Grid blob trigger.** The default polling blob trigger is not supported there. Create an Event Grid subscription on the container and set `source="EventGrid"` in the Python v2 decorator. See Tutorial: Trigger Azure Functions on blob containers using an event subscription.
- Keep each run short. Hand long work to a container.
- Make the function safe to run twice for the same file (for example, use a hash of the file as the result key).
- Limit how many runs call your model at the same time, or you will hit `429` errors.

Main cost considerations

Flex Consumption charges per execution and for the memory and time used (and for any "always ready" instances you choose to keep). Many functions at once can send more requests to your model than its quota allows, which causes 429 errors. Limit how many run at the same time.

Common beginner mistakes

- Doing slow, heavy work inside a function. Keep functions short.
- Using a function key as if it were user sign-in. A function key is not a login system.
- Putting storage keys in settings instead of using a managed identity.
- Writing handlers that break when the same event arrives twice. Make them safe to repeat.
- Using the default polling blob trigger on Flex Consumption. It is not supported there; use the Event Grid source.

Official documentation

- Azure Functions overview
- Flex Consumption plan

Sign-In, Roles and Secrets

Your application needs to prove who it is to every service it calls, and it needs somewhere safe to keep anything secret. Getting this right early prevents most "403 Forbidden" errors and avoids leaked keys. This chapter keeps to the basics a junior engineer needs: **managed identities and roles** (so you do not need passwords) and **Key Vault** (for the secrets you cannot avoid). Advanced security topics, such as private networking and enterprise governance, are listed in What to Learn Later.

Managed identities and roles

What it does

A **managed identity** is a sign-in that Azure creates and looks after for your app. Your app never sees a password. When your code runs on Azure (in a container app, a function app and so on), `ManagedIdentityCredential` asks Azure for a short-lived token automatically.

There are two kinds:

- **System-assigned:** tied to one resource and deleted with it.
- **User-assigned:** a separate resource you attach to one or more apps. This is the better choice for most projects, because you can create it and give it roles *before* the app exists, and the roles survive redeployments.

Having an identity is not enough. You must also give it a **role** on each resource it uses. This is **role-based access control (RBAC)**: a *role* (a set of allowed actions) is assigned to an *identity* (a user, group or managed identity) at a *scope* (a resource group, a resource and so on). Roles are additive.

Roles a junior AI Engineer assigns most often:

Service	Azure role
Foundry (models, agents)	**Foundry User** (build and use); **Foundry Agent Consumer** (only call agents)
Azure AI Search	**Search Index Data Reader** (query), **Search Index Data Contributor** (add and change data), Search Service Contributor (manage)
Blob Storage	**Storage Blob Data Reader** or **Contributor**
Cosmos DB	**Cosmos DB Built-in Data Reader** or **Contributor** (Cosmos DB has its own role system)
Key Vault	**Key Vault Secrets User**
Container Registry	**AcrPull** (run images), **AcrPush** (upload images)

> **Warning — Role names change**
> The Foundry roles were renamed from *Azure AI User*, *Owner*, *Account Owner and Project Manager* to *Foundry User*, *Owner*, *Account Owner and Project Manager*. Microsoft advises not to use roles starting with *Cognitive Services* for Foundry (except *Cognitive Services Usages Reader*), and not to use *Azure AI Developer* for Foundry projects.

When to use it

Give each application its own identity, and give it only the roles it needs on the narrowest scope that works. Give people roles through **groups** rather than one by one.

Set it up in the portal

1. **Create the identity. Create › Managed Identity.** Use a name per app and environment, such as `id-ragapi-dev`.
2. **Attach it to your app.** Open the Container App or Function App, go to **Identity › User assigned › Add**, and select it.
3. **Give it roles.** Open the target resource (for example your Foundry resource), choose **Access control (IAM) › Add › Add role assignment**, select the role, choose **Managed identity** as the member type, and pick your identity. Use **Check access** to see what an identity can do. Roles can take a few minutes to take effect.
4. **Test it.** From the running app, create a `ManagedIdentityCredential` and call the service. A `403` means a missing role or a role at the wrong scope.
5. **Clean up.** Remove role assignments before you delete an identity. Orphaned assignments show as "Identity not found".

Use it from code

Use the same code on your laptop and in Azure: DefaultAzureCredential uses your `az login` sign-in on a laptop and a managed identity in Azure. If you prefer to be explicit, choose `ManagedIdentityCredential` in Azure (with the client ID from `AZURE_CLIENT_ID` for a user-assigned identity) and `DefaultAzureCredential` elsewhere, based on configuration.

You can also give a role to an app's identity with the CLI instead of the portal. Assign Azure roles using Azure CLI shows `az role assignment create` with the identity, the role and the scope. Use the narrowest scope that works, for example one search service rather than the whole subscription.

Main cost considerations

The cost of getting this wrong is time: roles that are too narrow cause `403` errors, and roles that are too wide are a security risk. Write down which role each identity has and why.

Common beginner mistakes

- Giving an app *Owner* or *Contributor* because "it works". Use the specific *data* role instead.
- Forgetting that *management* roles and *data* roles are different (see Cloud Basics).
- Using one identity for every app.
- Putting a client secret in app settings when a managed identity would do.
- Expecting a new role to work instantly. Wait a few minutes.

Official documentation

- Managed identities for Azure resources
- What is Azure RBAC?
- AI + machine learning built-in roles
- DefaultAzureCredential reference

Azure Key Vault

What it does

Azure Key Vault is a locked box for **secrets**: API keys for outside services, passwords and connection strings. Your code reads a secret at run time, so it never has to sit in your source code or your Git history. Key Vault also stores encryption keys and certificates. App Service, Functions and Container Apps can read a secret for you through a **Key Vault reference**, using the app's managed identity.

When to use it

Store only what you cannot replace with an identity: the API key of a third-party model provider, a webhook secret, or a legacy connection string. If a service supports sign-in with a managed identity, use that instead and you will have nothing to store.

Set it up in the portal

1. **Create › Key Vault.** Pick a name, a region and the **Standard** tier. For the permission model choose **Azure role-based access control**. Keep **soft delete** on and enable **purge protection** for anything important.
2. **Give access.** Give yourself **Key Vault Secrets Officer** (to add and change secrets). Give your app's managed identity **Key Vault Secrets User** (to read).
3. **Add a secret. Secrets › Generate/Import.** Set an expiry date.
4. **Use it.** In a Container App you can add a secret of type *Key Vault reference* with the secret's URI and the identity.
5. **Fix problems.** `403 ForbiddenByRbac` means a missing role. `ForbiddenByConnection` means a network rule. "Secret not found" often means the wrong name or version.
6. **Clean up.** A deleted vault stays soft-deleted, and its name is reserved, until the retention period ends or it is purged. A vault with purge protection cannot be purged early.

Use it from code

The Quickstart – Azure Key Vault Python client library – manage secrets shows how to read a secret with `SecretClient` and `DefaultAzureCredential`.

Key points:

- Read each secret once at start-up and keep it in memory. Do not call Key Vault on every request.
- Never print or log secret values.
- If reading fails with `403`, the identity is missing a role that can read secrets.

Main cost considerations

Key Vault charges per 10,000 operations, plus extra for hardware-protected keys and certificate renewals. Cache secrets in memory so you do not pay for (or get throttled on) repeated reads.

Common beginner mistakes

- Printing or logging a secret value.
- Copying a secret into a pipeline variable or `.env` file that gets committed.
- Giving an app the *Key Vault Administrator* role. It only needs *Key Vault Secrets User*.
- Turning off purge protection on a production vault, so an accidental delete cannot be undone.
- Storing a secret when a managed identity would remove the need for one.

Official documentation

- Azure Key Vault overview

Monitoring Your App

When something goes wrong in an AI app, the error is rarely obvious. A slow answer, a wrong answer and a crash can all look the same to a user. **Monitoring** shows you what your app is doing: how many requests it gets, how long they take, which ones fail and how many tokens they use. Set it up from the first deployment.

Azure Monitor and Application Insights

What it does

Azure Monitor is the umbrella name for Azure's monitoring tools:

- **Metrics:** numbers over time that every Azure resource produces, such as token usage and throttled requests for Foundry, or query delay for AI Search.
- **Logs:** text records stored in a **Log Analytics workspace** and searched with a query language called **KQL**.
- **Application Insights:** the part that watches *your code*: requests, dependencies, exceptions and traces. Your Python app sends data to it using **OpenTelemetry**, an open standard for telemetry, through the package `azure-monitor-opentelemetry`.
- **Alerts:** messages sent to you (for example by email) when something crosses a limit.

What to watch in an AI app:

Signal	Examples
The basics	Number of requests, error rate, delay (the slowest 5% of requests is the useful number)
Model use	Input and output tokens per request, throttled (`429`) calls
Search	Search delay, how often a search returns nothing
Quality	Scores from your test set, thumbs-up and thumbs-down from users
Cost	Tokens × price per day

When to use it

From day one. Without per-request token and delay data, you cannot debug, budget or tune. Do not log full prompts that contain personal data. Keep an eye on log volume, because logs are billed by the amount you store.

Set it up in the portal

1. **Create a Log Analytics workspace** (one per environment is a good start).
2. **Create Application Insights** (choose *workspace-based* and select your workspace). Copy its **connection string** into your app's settings. Treat it as configuration, not as part of your source code.
3. **Send resource logs to the workspace.** On each resource (Foundry, AI Search, Storage, your Container Apps environment) open **Diagnostic settings** and send the main log categories and metrics to the workspace.
4. **Connect Application Insights to your Foundry project** so agent and model traces appear in the Foundry portal.
5. **Make an alert.** For example, an alert when the number of throttled requests or server errors goes above a limit, sent to your email through an *action group*.
6. **Clean up.** Delete test workspaces, because they bill for stored data.

Use it from code

Enable Azure Monitor OpenTelemetry shows how to send traces, metrics and logs from a Python app with `configure_azure_monitor`, using the connection string of your Application Insights resource.

Key points:

- Call `configure_azure_monitor` **once at start-up**, before you create other clients.
- Record the **token usage** of each model call (input and output tokens) with the request, so you can see cost per request.
- Do not log full prompts that contain personal data.

Search your data with KQL. In Application Insights **Logs** you can find the slowest requests, the failures and the tokens used per hour. Log queries in Azure Monitor explains how queries work.

Main cost considerations

Application Insights and Log Analytics charge **per GB of data stored**, with a free allowance, and extra for keeping data longer than the included period. Control the cost by logging at a sensible level (not every detail), by sampling when traffic is high and by setting a retention period on purpose.

Common beginner mistakes

- Logging full prompts and answers that contain personal data.
- Never looking at the logs until something breaks.
- Alerts with nobody who knows what to do about them. Write one line about the first thing to check.
- Leaving a test workspace running and paying for its data.
- Not recording token counts, so you cannot explain a cost increase.

Official documentation

- Azure Monitor overview
- Application Insights overview
- Log Analytics overview
- Enable Azure Monitor OpenTelemetry (Python)

Cost Basics and Safe Habits

Cloud bills surprise beginners more than anything else. A forgotten deployment, a loop that calls a model thousands of times or a search service left running over a weekend can cost real money. This chapter gives you the habits that prevent that.

Cost, quotas and limits

What it does

You need to understand three ideas.

1. How things are billed. An AI system mixes several pricing styles:

Style	Examples
Pay for what you use	Model tokens, Document Intelligence pages, Functions
Pay by the hour while it exists	Azure AI Search tiers, databases, provisioned model capacity
Pay by the second while replicas run	Container Apps on the Consumption plan (the Dedicated plan is billed differently; see Container Apps)
Pay for storing and moving data	Blob storage, Cosmos DB storage, log data, data leaving a region

2. Budgets. A **budget** sends you an alert when spending reaches a limit you set. Set one *before* you create your first resource.

3. Quotas. A **quota** limits how much you can use, for example tokens per minute for a model. A quota is not a bill: it protects you and the provider. If you reach it you get HTTP `429`. See Cloud Basics.

Habits every AI Engineer needs in code:

- **Retry with a growing wait** when you get `429` or a server error, and add a little randomness (jitter). Do not retry `400` errors, because the request itself is wrong.
- **Set a timeout** on every network call.
- **Limit how many calls run at once**, so you stay under your quota.
- **Cache** answers you have already produced, so repeated questions do not cost tokens again.
- **Cap loops** and tokens per request.

When to use it

Before the first resource: estimate tokens per request and requests per day, then set a budget. After every experiment: delete what you do not need.

Set it up in the portal

1. **Create a budget. Cost Management › Budgets › Add.** Choose the subscription or resource group, a monthly amount and alert thresholds such as 50, 80 and 100 percent, with your email as the recipient.
2. **Look at what you spend. Cost analysis** lets you group costs by resource and by tag (for example the `project` tag).
3. **Read the advice. Azure Advisor** lists cost recommendations such as idle resources.
4. **Check your quota.** Open the **Quotas** page, or the quota view in the Foundry portal, to see tokens per minute for each model and region, and ask for more if you need it.
5. **Estimate before you build.** Use the **Azure pricing calculator**.

Use it from code

Many beginner jobs process lots of items. Two habits keep you under the quota:

- **Limit concurrency.** Allow only a few model calls at the same time (for example with a semaphore).
- **Retry 429 with a growing wait.** Wait a little longer after each failure and add a small random delay (jitter), so many clients do not retry together. The OpenAI client does this for you up to its retry limit; see the openai-python repository.

A budget. Create one in the portal under **Cost Management › Budgets**, or with the CLI. Tutorial: Create and manage budgets shows both.

Main cost considerations

A simple estimate (no prices assumed). Replace the bracketed rates with current values from the official pricing pages for your region and model, and note the date.

Item	Monthly formula
Chat answers	requests × (input tokens × [input rate] + output tokens × [output rate]) ÷ 1,000,000
Query embeddings	requests × query tokens × [embedding rate] ÷ 1,000,000
Document embeddings	new or changed chunks × tokens per chunk × [embedding rate] ÷ 1,000,000
Search service	search units × hours × [hourly rate]
OCR	pages × [per-page rate]
App hosting	replicas × running time × [rate]
Logs	GB stored × [rate]

A worked example. A support assistant gets 10,000 questions a day (about 300,000 a month). Each question uses 2,500 input tokens (500 of instructions, 1,800 of retrieved text and 200 of question and history) and 350 output tokens. That is 750 million input tokens and 105 million output tokens a month. Sending three chunks instead of five removes about 720 input tokens per request, around 216 million tokens a month. This is usually the single biggest saving.

Ways to spend less: use a smaller model for easy steps; shorten prompts and retrieved text; cache repeated answers; send batch jobs through batch deployments (a 50% discount compared with Global Standard); delete unused deployments and test services; tag everything.

Common beginner mistakes

- No budget alert in a learning subscription or account.
- Forgetting things that bill while idle: search services, databases, provisioned model capacity.
- Retrying without a wait, so many clients hammer the service at once.
- Loops without a limit.
- Caching answers without thinking about who is allowed to see them. Do not show one user's cached answer to another user.

Official documentation

- Microsoft Cost Management overview
- Azure Quotas overview
- Azure pricing calculator

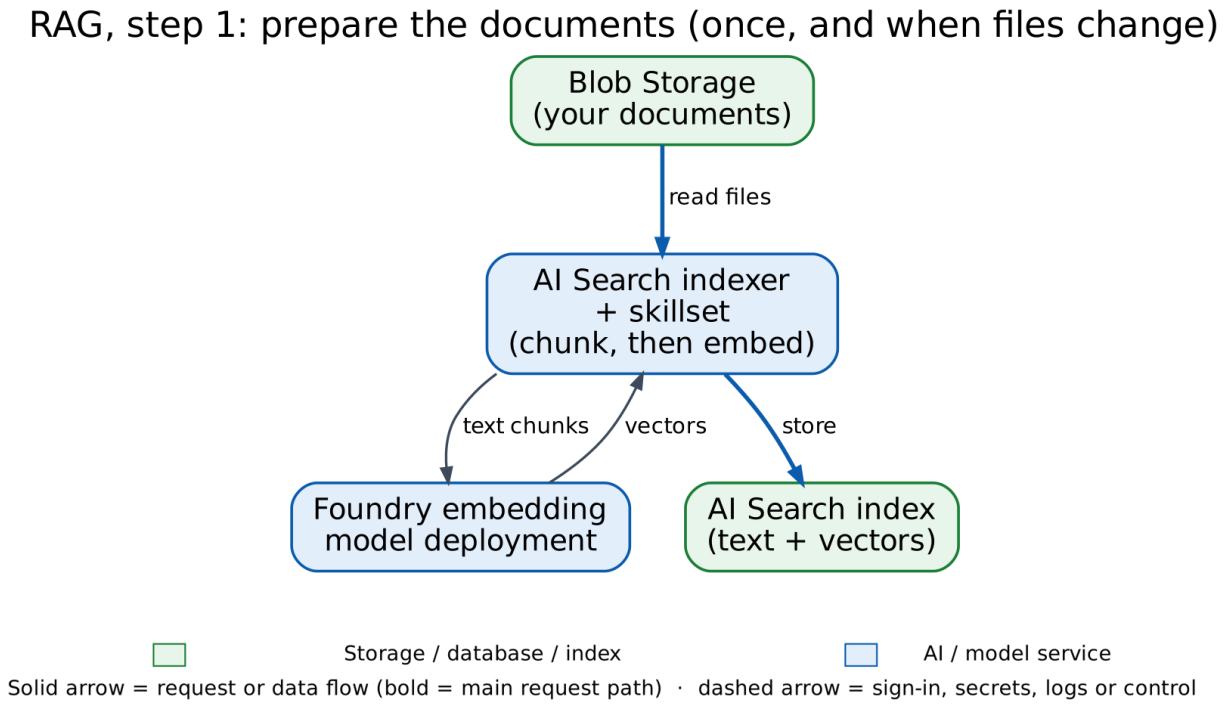
Four Starter Architectures

This chapter shows four small designs a junior AI Engineer can build: a RAG app, an AI agent app, an OCR pipeline and a Python AI API. Each design has two parts. The **beginner architecture** is the smallest set of services that shows the idea. The **production upgrades** are optional additions, each solving one specific problem. Add an upgrade only when you have that problem.

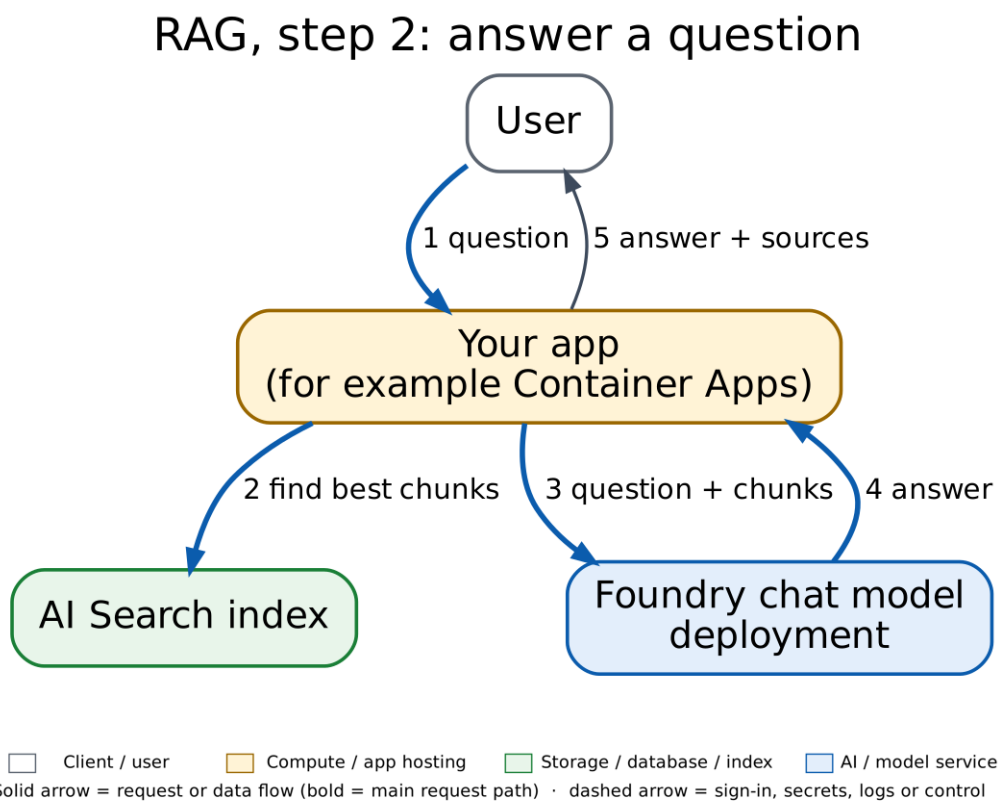
In every diagram, solid arrows are requests or data, bold arrows are the main path, and dashed arrows are sign-in or control. For every service, keep three questions in mind: what goes in, what comes out, and which role or setting lets it talk to the next service.

Architecture 1: Chat with your documents (RAG)

RAG has two separate flows, so it has two small diagrams. First you prepare the documents. Then you answer questions.



RAG step 1: files in Blob Storage are read by an AI Search indexer, whose skillset splits them into chunks and calls an embedding model deployment; the text and vectors are stored in an AI Search index.



RAG step 2: the user's question goes to your app, which searches the AI Search index for the best chunks, sends the question and chunks to a Foundry chat model deployment, and returns the answer with its sources.

How it works

Step 1: prepare the documents. Your files sit in Blob Storage. An **indexer** in AI Search reads them. A **skillset** attached to the indexer splits the text into chunks and calls an embedding model to turn each chunk into a vector. The indexer saves the text and vectors in the **index**. None of this happens by itself: the indexer, skillset and index must be configured. The **Import data** wizard in the portal creates all of them for you from your choices, and it uses fixed chunking settings (see Azure AI Search).

Step 2: answer a question. Your app receives the question, searches the index (a hybrid query is a good default), puts the best chunks into a prompt, asks the chat model to answer only from them, and returns the answer with the source names. Your app, not AI Search, calls the chat model.

What each service does

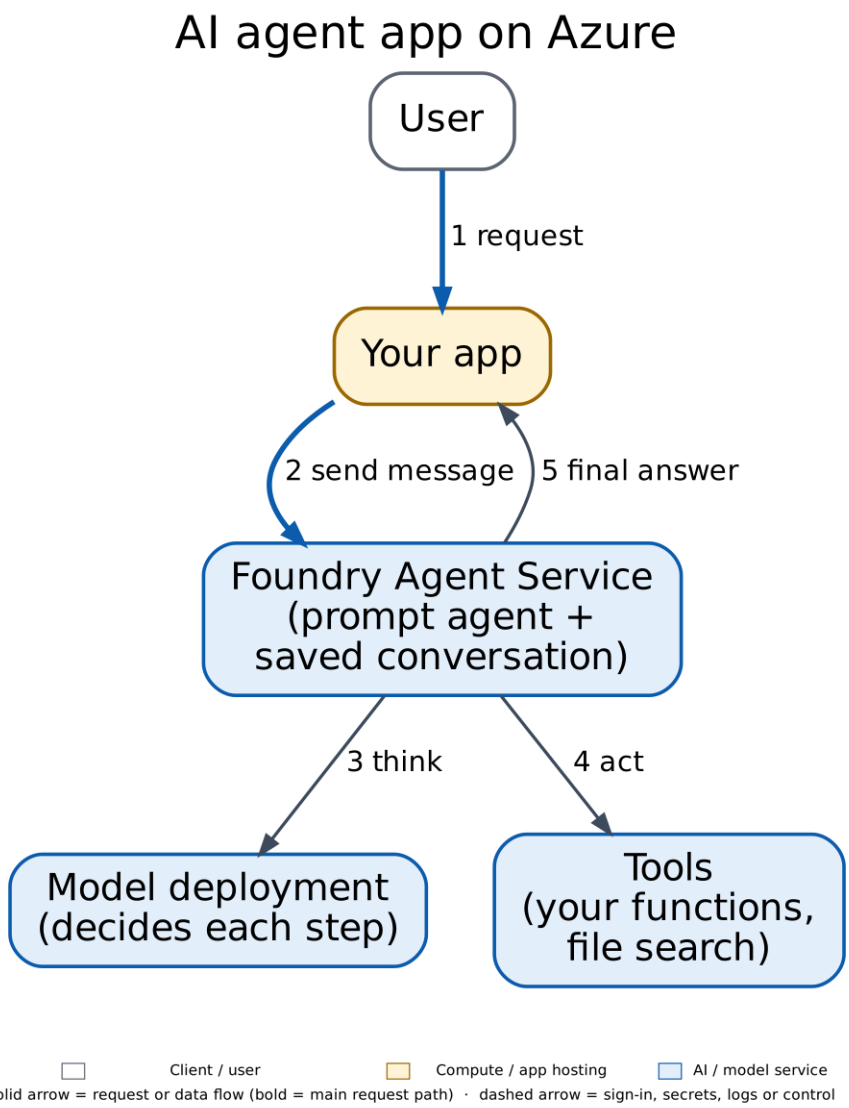
Service	What it does here	Essential?
Blob Storage	Holds the source files. Input: uploads. Output: files for the indexer.	Yes (or another supported source)
AI Search (indexer, skillset, index)	Chunks and embeds files, stores them, and finds the best chunks for a query.	Yes
Foundry embedding deployment	Turns text into vectors, at indexing time and for the query.	Yes
Foundry chat deployment	Writes the answer from the chunks you give it.	Yes
Your app	Takes the question, searches, builds the prompt, returns the answer. Can run on Container Apps or on your laptop while learning.	Yes

Production upgrades (optional)

Add this	When you have this problem
Content filters, Prompt Shields	Real users can type anything, or documents may hide instructions.
Cosmos DB for chat history	You want multi-turn conversations that survive restarts.
Document Intelligence	Your files are scans or have complex tables.
Application Insights	You need to see errors, delay and token use.
Managed identity and roles	Your app runs in Azure and must not use keys.
Private networking	A security team requires it (see What to Learn Later).

Cost drivers: tokens in the prompt (mostly the retrieved chunks), the search service (billed by the hour on dedicated tiers), embeddings when you index. **Things that go wrong:** an embedding model or vector size that does not match the index, 429 errors at busy times, files the indexer cannot read, and empty search results that lead the model to invent an answer. Always allow "I don't know". The answer step is described in RAG basics.

Architecture 2: AI agent app



Agent app: the user's message goes to your app, which sends it to a Foundry prompt agent; the agent uses a model deployment to decide each step and calls tools such as your functions or file search; the final answer returns through your app.

How it works

1. The user sends a request to your app.
2. Your app sends the message to the agent, in a **conversation**, with `responses.create` and the agent reference.
3. The agent's model decides what to do next.
4. If needed, the agent calls a **tool**, such as a function that looks up an order. The result goes back to the agent, which may repeat steps 3 and 4.
5. The agent returns the final answer, and the conversation keeps the history.

What each part does

Part	What it does here	Essential?
Your app	Receives the user's message and calls the agent.	Yes
Foundry Agent Service (prompt agent)	Holds the instructions, runs the loop and keeps the conversation.	Yes
Model deployment	Decides each step. It must support tool calling.	Yes
Tools	The actions the agent may take. Start with one.	Yes (at least one)

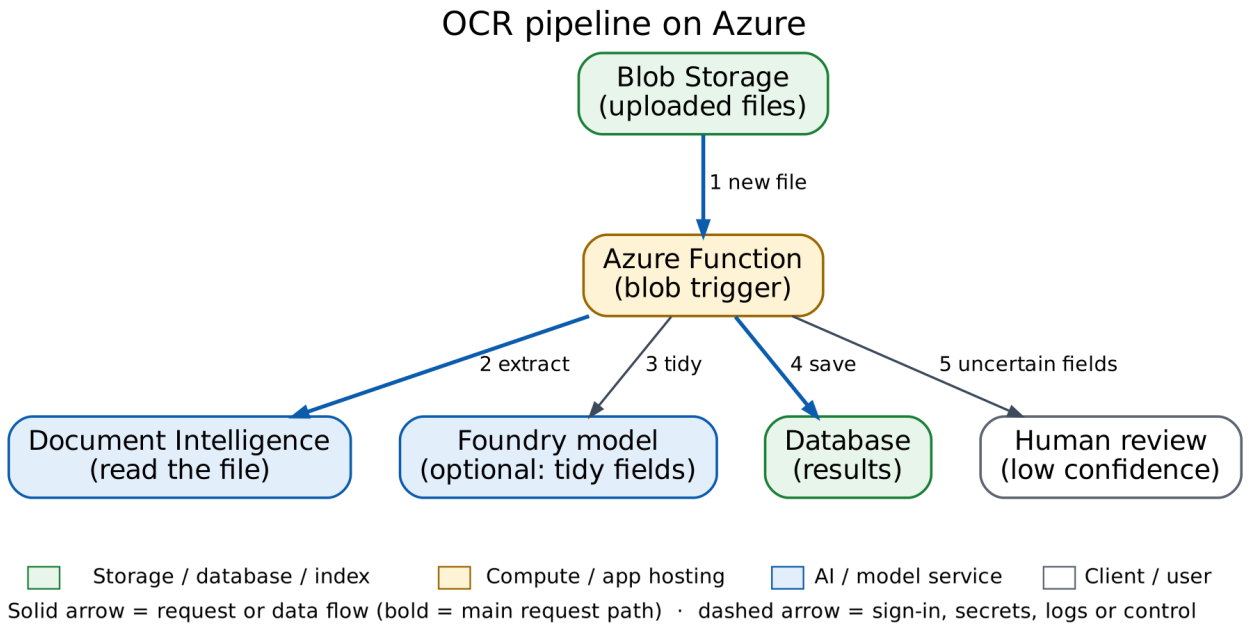
You also need the role that lets your app's identity use the project (for example *Foundry User*) and a **limit on loops and tokens**.

Production upgrades (optional)

Add this	When you have this problem
AI Search as a tool	The agent must answer from your documents.
Application Insights traces	You need to see every step of a run.
Confirmation before actions	A tool changes data or costs money.
Evaluation conversations	You change prompts or tools and need to avoid regressions.

Make each tool do one small thing and check its inputs. **Cost drivers:** several model calls per user message and tool charges. **Things that go wrong:** the agent picks the wrong tool, loops too long, a tool times out, a document tries to give the agent new orders, or an identity is missing a role (`401` or `403`).

Architecture 3: OCR pipeline



OCR pipeline: a file uploaded to Blob Storage triggers an Azure Function, which sends it to Document Intelligence, optionally asks a Foundry model to tidy the fields, saves the result in a database and sends uncertain fields to human review.

How it works

1. A person or app uploads a file to an `incoming` container.
2. Event Grid delivers a new-file event, which starts the function (on Flex Consumption the blob trigger must use the Event Grid source).
3. The function sends the file to Document Intelligence. For long documents this is an asynchronous job, so the function waits for the result.
4. Optionally, a model with **structured output** turns the extracted text into clean, checked fields.
5. The result is saved in a database (PostgreSQL or Cosmos DB).
6. Fields with a low confidence score go to a person for review.

Make the pipeline safe to repeat: use a hash of the file as the result key, so processing the same file twice does not create two records.

What each part does

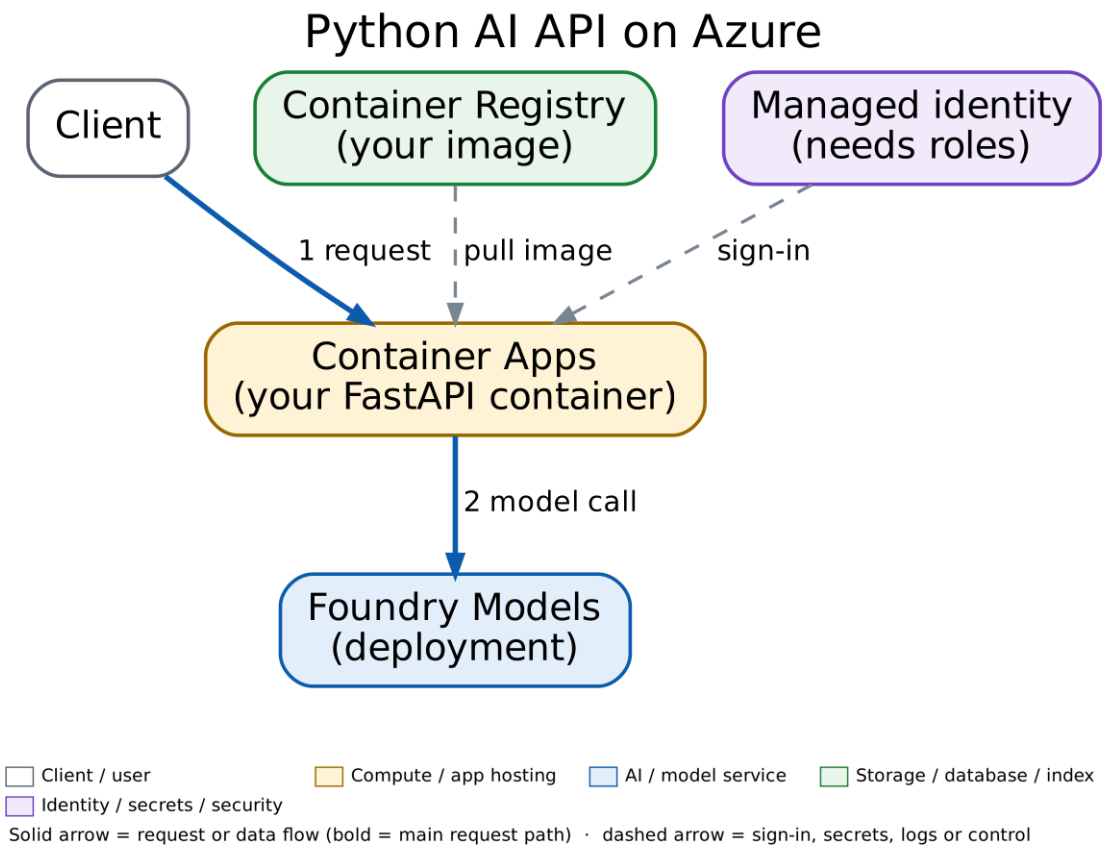
Part	What it does here	Essential?
Blob Storage	Receives uploads and raises the new-file event.	Yes
Azure Function	Runs your code for each file.	Yes
Document Intelligence	Reads text, tables and fields, with confidence scores.	Yes
Foundry model	Cleans and validates fields. Skip it if documents have one fixed layout.	Optional
Database	Stores results.	Yes
Human review	Checks low-confidence fields.	Strongly advised

Production upgrades (optional)

Add this	When you have this problem
A queue between upload and processing	Thousands of files arrive in bursts (see What to Learn Later).
Application Insights	You need to find failed files quickly.
Managed identity and roles	The function must reach storage and AI services without keys.

Cost drivers: pages processed, tokens for the clean-up step, and function time. **Things that go wrong:** unsupported or password-protected PDFs, very large files, throttling, and a function that does too much and times out.

Architecture 4: Python AI API



Python AI API: a client calls your FastAPI container running in Container Apps, which calls a Foundry Models deployment; the container image comes from Container Registry; a managed identity with the right roles provides sign-in.

How it works

You write a small web API (for example with FastAPI) that wraps a model call, put it in a container image and store the image in Container Registry. Container Apps runs the image. The app signs in to the model with its **managed identity**, so no keys are in the code. The identity needs a role on the model resource (for example *Foundry User*) and, to pull the image, *AcrPull* on the registry.

What each part does

Part	What it does here	Essential?
Container Apps	Runs and scales your container and gives it a web address.	Yes
Container Registry	Stores the image that Container Apps pulls.	Yes (for container deployment)
Foundry Models deployment	The model your API calls.	Yes
Managed identity	Lets the app sign in without a key. Needs role assignments.	Yes

Production upgrades (optional)

Add this	When you have this problem
Application Insights	You need logs, traces and alerts.
Key Vault	A third-party key cannot be avoided.
Minimum of one replica	A slow first request after idle time is not acceptable.
API Management	Several teams share model capacity (see What to Learn Later).

Start from Microsoft's quickstart linked in Container Apps. **Cost drivers:** the replicas you keep running, model tokens and log volume. **Things that go wrong:** a wrong port so health checks fail, a missing *AcrPull* role so the image will not pull, slow first requests when the app scales from zero, and 429 errors from the model under load.

Which services go together?

What you are building	Smallest Azure combination
Chat with your documents	Blob + AI Search + Foundry Models + your app
Customer-support agent	Foundry Agent Service + a function tool (add AI Search when it needs documents)
Invoice or form reading	Blob + Azure Functions + Document Intelligence + a database
A simple AI API	Container Apps + Container Registry + Foundry Models

Learning Roadmap for Juniors

Time estimates assume part-time study and some Python experience.

> **Best practice — Finish one project before starting the next**
> Build each project until it works and you have measured it, then move on. A small project that works and is measured teaches more than many half-finished ones.

Step 1: Cloud and Python basics (about 2 weeks)

Aspect	Detail
What to learn	The portal, the Azure CLI and `az login`; subscriptions, resource groups and regions; what a managed identity is and why roles matter; Python virtual environments, `async`, FastAPI and Pydantic; Docker basics.
What to practise	Create a resource group, create a resource, give yourself a role, delete the group. Read a `401`, a `403` and a `429` and say what each means.
Small project	Create a Foundry resource, deploy a chat model and call it from Python with `DefaultAzureCredential`. Log the token usage of each call.

Step 2: Build with models and RAG (about 4 to 6 weeks)

Aspect	Detail
What to learn	The Responses API: prompts, structured output, tool calling, streaming, embeddings; Blob Storage; Document Intelligence; Azure AI Search (hybrid search, filters); chunking; Content Safety basics; testing with a golden set.
What to practise	Writing a prompt that cites sources; fixing bad retrieval by changing chunk size and the number of results; measuring recall@5.
Small project	**"Policy Q&A".** Upload 20 to 50 PDFs to Blob Storage. Index them with the AI Search wizard. Build a FastAPI `/ask` endpoint that returns an answer with citations. Add a 30-question golden set and measure recall@5 and groundedness.
Result	A working RAG API with measured quality and a README that explains your choices.

Step 3: Deploy, watch and control cost (about 3 to 4 weeks)

Aspect	Detail
What to learn	Container Apps and Container Registry; managed identities with roles instead of keys; Key Vault; Application Insights; budgets and quotas; PostgreSQL with pgvector; Cosmos DB for chat history.
What to practise	Containerising your API; deploying it with a managed identity; reading logs; setting a budget alert; deleting everything you do not need.
Small project	Deploy "Policy Q&A" to Container Apps. Add chat history in Cosmos DB, send traces to Application Insights, and write down the cost of 1,000 questions using the formulas in Cost Basics.
Result	A deployed, key-free API with a budget and a cost estimate.

Step 4: Agents and OCR (about 3 to 4 weeks)

Aspect	Detail
What to learn	Foundry prompt agents and function tools; Azure Functions with a blob trigger; extracting fields with Document Intelligence and a model with structured output; confidence scores and human review.
What to practise	Capping an agent loop; testing a tool with bad input; making a pipeline safe to run twice.
Small projects	An **order-status agent** with one function tool. An **invoice reader**: upload an invoice, extract the vendor, date and total, save them, and flag low-confidence fields.
Result	One agent and one OCR pipeline you can demonstrate and explain.

Certifications (optional)

Certifications do not replace projects, but some employers use them as filters.

> Not fully verified **— Certification names and dates**
>
> AI-102 (Azure AI Engineer Associate) retired on 30 June 2026. Third-party sources name its replacement as **AI-103**, *Azure AI App and Agent Developer Associate*, with the exam title *Developing AI Apps and Agents on Azure*. Check the official catalogues before you register: Microsoft credentials.

What to Learn Later

This guide left out topics that are valuable but not needed to ship your first AI projects. This short chapter lists them, explains why each matters and shows when you will probably need it. Do not try to learn these now. Come back when a real project or a new role asks for them.

Topic	What it is and why it matters	You will need it when
Private networking	Keeping services off the public internet with virtual networks, private endpoints and private DNS. Often required by a company's security team.	Your app handles private or regulated data.
Permissions on documents (security trimming)	Making sure each user only retrieves documents they are allowed to read, by saving permissions with each chunk and filtering every search.	You build RAG over documents with different access rights.
Enterprise governance	Policies, management groups, central logging, approval flows and key rotation across many teams.	You join a larger organisation with a platform team.
API gateways (Azure API Management)	One front door for model calls with token limits, usage tracking and failover between deployments.	Several teams share model capacity.
Queues and events (Service Bus, Event Grid, Event Hubs)	Messaging between parts of a system so bursts and failures are handled safely.	Your OCR pipeline must process thousands of files reliably.
Provisioned throughput	Reserved model capacity for steady high traffic and stable delay, billed by the hour.	Pay-per-token costs or delays become a problem at scale.
Data engineering (Data Factory, Fabric, Databricks)	Building large data pipelines and warehouses.	Your AI needs data prepared at scale.
Machine learning platforms (Azure Machine Learning)	Training, tracking, registering and hosting your own models.	You need a custom model, not a ready-made one.
Fine-tuning	Adjusting a model's behaviour or format with your own examples.	Prompting and RAG cannot give you the style or format you need.
Virtual machines and GPUs	Servers you manage yourself, including GPU machines.	You self-host a model or train one.
Kubernetes (AKS)	Running many containers with full control, including GPU scheduling.	A platform team runs your workloads.
Hosted agents and multi-agent systems	Packaging your own agent framework code and making agents work together.	Prompt agents cannot do what you need.
Agentic retrieval and Foundry IQ (knowledge bases)	An LLM-guided retrieval layer over several knowledge sources, built on Azure AI Search.	A single hybrid search is not enough for complex questions.
Speech, Vision, Language, Translator	Prebuilt services for audio, images, text analysis and translation.	Your product handles voice, images or many languages.
Infrastructure as code (Bicep, Terraform)	Describing cloud resources in files so they can be created the same way every time.	You work in a team that deploys through pipelines.
CI/CD (GitHub Actions with OIDC)	Building, testing and deploying automatically, without stored passwords.	You release changes regularly.
MLOps	Pipelines, registries and monitoring for model lifecycles.	You run custom models in production.
High availability and disaster recovery	Surviving the loss of a zone or region, with backups and failover.	Users depend on your service around the clock.
Caching and semantic caching (Azure Managed Redis)	Storing answers or embeddings to save tokens and time.	Cost or delay grows with traffic.
Edge, sovereign and government clouds	Running AI on devices, or in clouds with special rules.	A customer or law requires it.

Where to start when you are ready

Topic	Official starting point
Private endpoints	Private endpoints
API Management as an AI gateway	AI gateway in API Management
Azure Machine Learning	Azure Machine Learning
Fine-tuning in Foundry	Fine-tuning in Foundry
GPUs and Kubernetes	GPUs on AKS and KAITO on AKS
Infrastructure as code	Bicep
Deploying from GitHub without passwords	GitHub OIDC to Azure
Agent frameworks	Microsoft Agent Framework
Designing AI workloads well	Azure Well-Architected Framework for AI

Quick Reference

Short tables for daily use. For the full comparison of each service see the Service Map. For ready-made combinations see Starter Architectures.

Which service should I use?

If you need...	Choose
To call an LLM with no servers to run	Foundry Models (Global Standard)
Cheap processing that can wait a day	Global Batch deployment
An agent with tools and no hosting code	Foundry prompt agent
Search with keywords and meaning	Azure AI Search
RAG indexing set up by a wizard (you still review the objects it creates)	AI Search Import data wizard
Cheap vector storage, next to your app data	PostgreSQL with pgvector
OCR, tables and forms	Document Intelligence
Blocking harmful content and prompt injection	Content filters, Prompt Shields, Content Safety
Files and documents	Blob Storage
Chat history	Cosmos DB
A containerised API	Container Apps
A small event-driven function	Azure Functions
To avoid passwords and keys	Managed identity with roles
To store a third-party key	Key Vault
Logs, traces and alerts	Application Insights, Azure Monitor
Spending alerts	Cost Management budgets

Python packages and imports

Task	pip package → import
Sign in	`azure-identity` → `from azure.identity import DefaultAzureCredential, ManagedIdentityCredential, get_bearer_token_provider`
Call a model	`openai` → `from openai import OpenAI` (base URL ends in `/openai/v1/`)
Foundry project and agents	`azure-ai-projects` → `from azure.ai.projects import AIProjectClient`; `from azure.ai.projects.models import PromptAgentDefinition, FunctionTool`
Search	`azure-search-documents` → `SearchClient`, `SearchIndexClient`, `VectorizableTextQuery`
OCR	`azure-ai-documentintelligence` → `DocumentIntelligenceClient`
Safety	`azure-ai-contentsafety` → `ContentSafetyClient`
Evaluation	`azure-ai-evaluation` → `evaluate`, `GroundednessEvaluator`
Files	`azure-storage-blob` → `BlobServiceClient`
NoSQL	`azure-cosmos` → `CosmosClient`
PostgreSQL	`psycopg`, `pgvector`

Task	pip package → import
Secrets	`azure-keyvault-secrets` → `SecretClient`
Logs and traces	`azure-monitor-opentelemetry` → `configure_azure_monitor`

Ways to sign in

Method	Use it when	Notes
`DefaultAzureCredential`	Local development	Uses your `az login` sign-in on a laptop. In a deployed app, prefer `ManagedIdentityCredential`, or set `AZURE_TOKEN_CREDENTIALS=prod` so `DefaultAzureCredential` only tries production credentials.
Managed identity (`ManagedIdentityCredential`)	Your code runs in Azure, for example in Container Apps or Functions	Preferred for deployed apps. No secrets to store. Set `AZURE_CLIENT_ID` for a user-assigned identity. Give it only the roles it needs.
Entra token for PostgreSQL	You connect to Azure Database for PostgreSQL	Pass a token from `DefaultAzureCredential` as the password, so no password is stored (see PostgreSQL and pgvector). Tokens expire, so get a new one when you connect.
API key	A tool cannot use tokens, or a quick test	Keep it in Key Vault. Never in code or Git.
SAS token	You share one file for a short time	Make it last minutes, not days.

Everyday commands

The commands you will use most often. Each one is documented in the linked Microsoft page.

Task	Azure CLI command	Official page
Sign in and choose a subscription	`az login`, `az account set`	Sign in with Azure CLI
Create or delete a resource group	`az group create`, `az group delete`	Manage resource groups with Azure CLI
See models and deployments	`az cognitiveservices model list`, `az cognitiveservices account deployment list`	CLI reference
Check or give roles	`az role assignment list`, `az role assignment create`	Assign Azure roles using Azure CLI
Build an image in the cloud	`az acr build`	Build a container image on demand

Common errors and fixes

Symptom	Likely cause	Fix
401 Unauthorized	Wrong token scope, wrong or expired key, key sign-in turned off	Use `https://ai.azure.com/.default` for v1 model calls; sign in with Entra ID; use the custom-domain endpoint
403 Forbidden	Missing *data* role, role not yet active, or a network rule	Assign Foundry User, Search Index Data Reader or Storage Blob Data Reader; wait a few minutes
404 DeploymentNotFound	The deployment name in code differs from the portal, or it is still being created	List deployments; keep the name in configuration
429 Too Many Requests	Quota used up	Retry with a growing wait; send fewer calls at once; ask for more quota
400 with a content-filter code	Input or output was blocked	Read the error body; change the prompt or the filter; do not retry unchanged

Symptom	Likely cause	Fix
AI Search indexer fails	Unsupported file, time-out or embedding throttling	Read the indexer run history; raise the embedding quota
Made-up answers	Search found nothing useful, or the prompt allows guessing	Measure recall@k; use hybrid search; require citations and "I don't know"
Container app will not start	Image pull failed (missing AcrPull), wrong port, health check failing	Read the system logs; fix the role and the port
Function trigger does not fire	Wrong connection settings, a missing role, or a polling blob trigger on Flex Consumption	Check the `<CONNECTION>__*` settings and the roles; use the Event Grid source and an event subscription
Cosmos DB 429	RU/s used up	Query with the partition key; raise RU/s or use autoscale
Cost suddenly rises	Looping agent, leaked key, full re-index, idle paid resources	Check token metrics, set budgets, cap loops, rotate keys, delete idle resources

Where to run your code

Option	Good for	Watch out for	Typical use
Your laptop (local development)	Learning, debugging, first versions	Uses your own sign-in, not an app identity; not for real users	Trying prompts, building a first RAG version
Azure Functions	Event triggers, pay per run, nothing to manage	Time limits, slow first run, large packages	Reacting to uploads, small tools
Container Apps	Any container, scaling, safe releases	Slow first request if scaled to zero	RAG and agent APIs
Foundry prompt agent (no hosting code)	An agent defined by instructions, a model and tools	Less control than your own code; each change is a new version	A first business agent

What drives cost, and what to do about it

Cost driver	What to do
Input tokens (long prompts, retrieved text, chat history)	Send fewer and better chunks; summarise long history
Output tokens	Limit the answer length; use structured output instead of long text
Model choice	Use a small model for easy steps; compare models on your golden set
Work that can wait	Use batch deployments or batch inference
Idle paid resources (search tiers, databases, provisioned capacity)	Delete or stop them when you are not using them
Re-indexing	Only re-process changed files; reuse embeddings
OCR	Use the simplest model that works; skip files you already processed
Logs	Log at a sensible level; set retention on purpose
Data leaving a region	Keep services in the same region

Glossary

Plain-language definitions of the terms used in this guide, in alphabetical order within each group.

Cloud basics

Term	Meaning
Control (management) access	Permission to create, change or delete a resource, as opposed to using it.
Data access	Permission to use a resource: call a model, query an index, read a blob. On Azure this often needs its own role.
Quota	A limit on how much of a service you may use, for example tokens per minute.
Region	The geographic location where a resource runs.
Resource	Anything you create in a cloud: a model deployment, a database, a storage account.
Resource group	A folder for related resources. Deleting it deletes everything inside.
Subscription	The Azure unit for billing and quotas.
Tag	A name and value attached to a resource, used to find it and track its cost.

AI and models

Term	Meaning
Agent	A program in which a model decides, step by step, which tool to use next to reach a goal.
Agent version	A saved, numbered definition of a Foundry agent.
Chunk	A short passage of a document, small enough to be searched and put in a prompt.
Confidence score	A number saying how sure an OCR or extraction service is about a value.
Context window	The maximum amount of text (in tokens) a model can read in one request.
Conversation (Foundry)	The saved history of an exchange with an agent.
Deployment (Foundry)	A named copy of a model you can call, with a type and a capacity.
Embedding	A list of numbers that captures the meaning of a text, used for search by meaning.
Golden set	A fixed list of test questions with known good answers or source documents.
Groundedness	How well an answer is supported by the sources given to the model.
Guardrail / content filter	A check that blocks harmful or unwanted input or output.
Hallucination	A fluent answer that is not supported by facts or sources.
Hybrid search	Search that combines keyword matching and search by meaning.
Inference	Running a model to get an answer.
LLM	Large language model.
MCP (Model Context Protocol)	A standard way to expose tools to agents.
OCR	Optical character recognition: reading text from images and scans.
Prompt	The instructions and input you send to a model.
Prompt injection	Hidden instructions in a document or web page that try to take control of the model.
RAG	Retrieval-augmented generation: finding relevant passages and giving them to a model so it can answer from them.
recall@k	The share of test questions for which a correct document appears in the top k search results.
Reranking / semantic ranker	A second step that re-orders the top search results to put the best first.

Term	Meaning
Responses API	The OpenAI API style recommended for new work, with conversations and tools.
Structured output	A model answer that follows a JSON schema, so code can use it safely.
Token	A small piece of text (part of a word) that models read and write. Pricing and quotas are counted in tokens.
Tool (function) calling	The model asks for a function to be run, and your code runs it and returns the result.
Vector	The list of numbers in an embedding.
Vector search	Finding the stored vectors closest in meaning to a query vector.

Data and storage

Term	Meaning
Blob / container / storage account	A file, a folder-like group of files, and the account that holds them.
Index (search)	The structure that stores searchable content.
Indexer	An Azure AI Search feature that pulls files from a source and indexes them.
Integrated vectorization	Chunking and embedding done by the search service while indexing.
Partition key	The field that decides how Cosmos DB spreads data. Queries that use it are fast.
pgvector	A PostgreSQL extension for storing and searching vectors.
Request Unit (RU)	The unit Cosmos DB uses to measure the cost of an operation.
SAS (shared access signature)	A temporary link that gives limited access to Azure Storage data, such as one file.
TTL	Time to live: the time after which data is deleted automatically.

Sign-in and security basics

Term	Meaning
Credential chain	The ordered list of places an SDK looks for credentials, using the first that works.
Entra ID	The Microsoft identity service (formerly Azure Active Directory).
Key Vault	The Azure service for storing secrets such as API keys and passwords.
Least privilege	Giving only the permissions that are needed.
Managed identity	A sign-in that Azure creates and manages for your app, with no password for you to handle.
RBAC	Role-based access control: giving roles to identities at a scope.
Role	A named set of permissions.
Secret	A value that must stay private, such as a key or password.

Running and watching your app

Term	Meaning
Application Insights	The Azure tool that records requests, errors and traces from your code.
Container	A package of your code and everything it needs to run.
Container image / registry	The file a container starts from, and the private store that holds it (Azure Container Registry).
Ingress	The setting that gives a container app a web address.
Jitter	A small random delay added to retries so many clients do not retry at the same moment.
KQL	Kusto Query Language, used to search Azure logs.
Log Analytics workspace	Where Azure Monitor stores logs.
OpenTelemetry	An open standard for sending traces, metrics and logs.
Replica	One running copy of your app.
Revision	An unchangeable version of a container app.
Trigger	The event that starts a function: a web request, a timer, a new file or a message.

Coverage, Exclusions and Open Items

This guide does **not** claim to cover all of Azure. Its scope is the junior AI Engineer path described in Scope and verification. Links were opened on 12 and 13 October 2026. A working link does not prove a claim is correct. On 13 October 2026 the claims listed in Scope and verification were re-checked against Microsoft Learn. Most code was removed in favour of links to Microsoft's quickstarts; the three remaining snippets match their linked pages and were not executed.

What is covered

Each service section above has: what it does, when to use it, portal steps, the key ideas for using it from code (with links to Microsoft's quickstarts and SDK samples), main cost considerations, common mistakes and official links. The Service Map lists all 18 sections.

Items not fully verified

Item	Status
Code snippets	Only three short snippets remain (Azure CLI sign-in and resource group, DefaultAzureCredential, first model call). Each matches the linked Microsoft page; none was executed for this guide.
Linked quickstarts	Code in the linked Microsoft quickstarts and SDK samples was not reviewed line by line. Package versions and parameters there may change.
Agent Service details	Preview and general-availability status of some Agent Service features was not independently verified.
Model sign-in role	The v1 API page says Entra ID sign-in needs the Cognitive Services OpenAI User role, while this guide recommends Foundry User for Foundry resources. Check the role for your resource type. Microsoft's pages differ. The Foundry RBAC page recommends Foundry User and advises against roles starting with Cognitive Services, while the hosted-agent permissions page lists Cognitive Services OpenAI User for calling account-level OpenAI endpoints directly. If Foundry User returns 403 on the /openai/v1/ endpoint, check the current role guidance for your resource type.
PostgreSQL networking and sign-in steps	The `azure.extensions` allowlist and `CREATE EXTENSION vector` steps were checked. The public-access firewall-rule and Entra-admin portal steps were not re-checked.
Certifications	Exam names come from several non-Microsoft sources; check Microsoft's credential pages.
Prices	No absolute prices are quoted; use the pricing pages.
Regional availability	Model and feature availability by region is not tabulated; it changes often.
Preview features	Anything labelled preview may change or be withdrawn.
Links	Each external link was opened on 12 and 13 October 2026 (see the Documentation Index). Re-check before relying on one.

Exclusions

- AWS and other clouds (this edition covers Azure only).
- Everything in What to Learn Later (private networking, enterprise governance, data engineering, VMs, Kubernetes, MLOps, fine-tuning).
- Full code samples (each section links to Microsoft's quickstart or SDK sample instead).
- Non-Python SDKs.
- Absolute prices and per-region availability tables.
- Sovereign and government clouds; edge and on-device AI.

Documentation Index

Every external link used in this guide, grouped by source. *Verified* means the page was opened on 12 or 13 October 2026 and its title matched the topic (the title is shown). This checks that the link works, not that every claim on the page or in this guide is correct. Links can move; if one fails, search the title on the provider's documentation site.

Microsoft Learn documentation

Resource	Page title at verification / URL	Used in
Access tiers for blob data	Access tiers for blob data learn.microsoft.com/en-us/azure/storage/blobs/access-tiers-overview	Storage and Databases
AI + machine learning built-in roles	Azure built-in roles for AI + machine learning learn.microsoft.com/en-us/azure/role-based-access-control/built-in-roles/ai-machine-learning	Sign-In, Roles and Secrets
AI gateway in API Management	AI gateway learn.microsoft.com/en-us/azure/api-management/genai-gateway-capabilities	What to Learn Later
Application Insights overview	Introduction to Application Insights learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview	Monitoring Your App
Assign Azure roles using Azure CLI	Assign Azure roles using Azure CLI learn.microsoft.com/en-us/azure/role-based-access-control/role-assignments-cli	Running Your Python App; Sign-In, Roles and Secrets; Quick Reference
Azure Blob storage trigger for Azure Functions	Azure Blob storage trigger for Azure Functions learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-storage-blob-trigger	Running Your Python App
Azure Cosmos DB resource model	Azure Cosmos DB resource model learn.microsoft.com/en-us/azure/cosmos-db/resource-model	Storage and Databases
Azure Machine Learning	What is Azure Machine Learning? learn.microsoft.com/en-us/azure/machine-learning/overview-what-is-azure-machine-learning	What to Learn Later
Azure OpenAI in Microsoft Foundry Models v1 API	API lifecycle learn.microsoft.com/en-us/azure/foundry/openai/api-version-lifecycle	Models and the Foundry Platform
Azure Quotas overview	Quotas overview learn.microsoft.com/en-us/azure/quotas/quotas-overview	Cost Basics and Safe Habits
Azure Well-Architected Framework for AI	AI workloads on Azure learn.microsoft.com/en-us/azure/well-architected/ai/get-started	What to Learn Later
Bicep	What is Bicep? learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/overview	What to Learn Later
Bicep quickstart	Bicep quickstart learn.microsoft.com/en-us/azure/foundry/how-to/create-resource-template	Models and the Foundry Platform
Build a container image on demand in Azure	Build a Container Image On-Demand in Azure learn.microsoft.com/en-us/azure/container-registry/container-registry-quickstart-task-cli	Running Your Python App; Quick Reference
Built-in RBAC roles for Microsoft Foundry	Built-in RBAC roles learn.microsoft.com/en-us/azure/foundry/concepts/rbac-foundry	Models and the Foundry Platform

Resource	Page title at verification / URL	Used in
Chunk large documents	Chunk large documents learn.microsoft.com/en-us/azure/search/vector-search-how-to-chunk-documents	Search and RAG
CLI reference	CLI reference learn.microsoft.com/en-us/cli/azure/cognitiveservices/account	Models and the Foundry Platform; Quick Reference
Create a user delegation SAS	Create a user delegation SAS learn.microsoft.com/en-us/rest/api/storageservices/create-user-delegation-sas	Storage and Databases
Create a vector query	Create a Vector Query learn.microsoft.com/en-us/azure/search/vector-search-how-to-query	Search and RAG
Create Foundry projects	Create projects learn.microsoft.com/en-us/azure/foundry/how-to/create-projects	Models and the Foundry Platform
Credential chains in the Azure Identity library for Python	Credential chains in the Azure Identity library for Python learn.microsoft.com/en-us/azure/developer/python/sdk/authentication/credential-chains	Cloud Basics
Design and develop a RAG solution	Design and develop a RAG solution learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-solution-design-and-evaluation-guide	Search and RAG
Enable and use pgvector	Enable and use pgvector learn.microsoft.com/en-us/azure/postgresql/flexible-server/how-to-use-pgvector	Storage and Databases
Enable Azure Monitor OpenTelemetry	Enable Azure Monitor OpenTelemetry learn.microsoft.com/en-us/azure/azure-monitor/app/opentelemetry-enable?tabs=python	Monitoring Your App
Fine-tuning in Foundry	Fine-tune (SFT) learn.microsoft.com/en-us/azure/foundry/openai/how-to/fine-tuning	What to Learn Later
Flex Consumption plan	Flex Consumption plan hosting learn.microsoft.com/en-us/azure/azure-functions/flex-consumption-plan	Running Your Python App
Function calling	Function calling learn.microsoft.com/en-us/azure/foundry/agents/how-to/tools/function-calling	AI Agents and Content Safety
Generate embeddings	Generate embeddings with Azure OpenAI learn.microsoft.com/en-us/azure/foundry/openai/how-to/embeddings	Models and the Foundry Platform
GitHub OIDC to Azure	Use the Azure Login action with OpenID Connect learn.microsoft.com/en-us/azure/developer/github/connect-from-azure-openid-connect	What to Learn Later
GPUs on AKS	Use GPUs on AKS learn.microsoft.com/en-us/azure/aks/gpu-cluster	What to Learn Later
How to use function calling	How to use function calling with Microsoft Foundry Models learn.microsoft.com/en-us/azure/foundry/openai/how-to/function-calling	Models and the Foundry Platform
How to use structured outputs	How to use structured outputs with Azure OpenAI in Microsoft Foundry Models learn.microsoft.com/en-us/azure/foundry/openai/how-to/structured-outputs	Models and the Foundry Platform
Hybrid search overview	Hybrid search learn.microsoft.com/en-us/azure/search/hybrid-search-overview	Search and RAG

Resource	Page title at verification / URL	Used in
KAITO on AKS	Deploy an AI model on AKS with the AI toolchain operator add-on learn.microsoft.com/en-us/azure/aks/ai-toolchain-operator	What to Learn Later
Learn	What is Microsoft Foundry learn.microsoft.com/en-us/azure/foundry/what-is-foundry	Service Map; Models and the Foundry Platform
Learn	Deployment types learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/deployment-types	Service Map; Models and the Foundry Platform
Learn	Agent Service overview learn.microsoft.com/en-us/azure/foundry/agents/overview	Service Map; AI Agents and Content Safety
Learn	What is Azure AI Content Safety? learn.microsoft.com/en-us/azure/ai-services/content-safety/overview	Service Map; AI Agents and Content Safety
Learn	What is Azure Document Intelligence in Foundry Tools? learn.microsoft.com/en-us/azure/ai-services/document-intelligence/overview	Service Map; OCR and Document Extraction
Learn	What is Azure AI Search? learn.microsoft.com/en-us/azure/search/search-what-is-azure-search	Service Map; Search and RAG
Learn	Observability in generative AI learn.microsoft.com/en-us/azure/foundry/concepts/observability	Service Map; Search and RAG
Learn	Introduction to Azure Blob Storage learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction	Service Map; Storage and Databases
Learn	What is Azure Database for PostgreSQL flexible server? learn.microsoft.com/en-us/azure/postgresql/flexible-server/overview	Service Map; Storage and Databases
Learn	Azure Cosmos DB learn.microsoft.com/en-us/azure/cosmos-db/introduction	Service Map; Storage and Databases
Learn	Azure Container Apps overview learn.microsoft.com/en-us/azure/container-apps/overview	Service Map; Running Your Python App
Learn	Introduction to Azure Container Registry learn.microsoft.com/en-us/azure/container-registry/container-registry-intro	Service Map; Running Your Python App
Learn	What is Azure Functions? learn.microsoft.com/en-us/azure/azure-functions/functions-overview	Service Map; Running Your Python App
Learn	What is managed identities for Azure resources? learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview	Service Map; Sign-In, Roles and Secrets
Learn	About Azure Key Vault learn.microsoft.com/en-us/azure/key-vault/general/overview	Service Map; Sign-In, Roles and Secrets
Learn	Azure Monitor overview learn.microsoft.com/en-us/azure/azure-monitor/overview	Service Map; Monitoring Your App

Resource	Page title at verification / URL	Used in
Learn	What is Microsoft Cost Management learn.microsoft.com/en-us/azure/cost-management-billing/costs/overview-cost-management	Service Map; Cost Basics and Safe Habits
Log Analytics overview	Overview of Log Analytics learn.microsoft.com/en-us/azure/azure-monitor/logs/log-analytics-overview	Monitoring Your App
Log queries in Azure Monitor	Log queries in Azure Monitor learn.microsoft.com/en-us/azure/azure-monitor/logs/log-query-overview	Monitoring Your App
Manage Azure resource groups by using Azure CLI	Manage Resource Groups - Azure CLI learn.microsoft.com/en-us/azure/azure-resource-manager/management/manage-resource-groups-cli	Cloud Basics; Quick Reference
Microsoft Agent Framework	Microsoft Agent Framework learn.microsoft.com/en-us/agent-framework/overview/agent-framework-overview	What to Learn Later
Microsoft credentials	Browse Credentials learn.microsoft.com/en-us/credentials/browse/	Learning Roadmap for Juniors
Private endpoints	What is a private endpoint? learn.microsoft.com/en-us/azure/private-link/private-endpoint-overview	What to Learn Later
Prompt Shields concepts	Prompt Shields learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/jailbreak-detection	AI Agents and Content Safety
Quickstart – Azure Key Vault Python client library – manage secrets	Quickstart – Azure Key Vault Python client library – manage secrets learn.microsoft.com/en-us/azure/key-vault/secrets/quick-create-python	Sign-In, Roles and Secrets
Quickstart: Analyze text content	Quickstart: Analyze text content learn.microsoft.com/en-us/azure/ai-services/content-safety/quickstart-text	AI Agents and Content Safety
Quickstart: Build and deploy your app from your local file system to Container Apps	Quickstart: Build and Deploy Your App from Your Local File System to Container Apps learn.microsoft.com/en-us/azure/container-apps/quickstart-code-to-cloud	Running Your Python App
Quickstart: Get started with Microsoft Foundry SDK	Quickstart learn.microsoft.com/en-us/azure/foundry/quickstarts/get-started-code	Models and the Foundry Platform; AI Agents and Content Safety
Quickstart: Prompt Shields	Quickstart: Prompt Shields learn.microsoft.com/en-us/azure/ai-services/content-safety/quickstart-jailbreak	AI Agents and Content Safety
Semantic ranking	Semantic ranking learn.microsoft.com/en-us/azure/search/semantic-search-overview	Search and RAG
Sign in with Azure CLI	Sign in with Azure CLI learn.microsoft.com/en-us/cli/azure/authenticate-azure-cli	Cloud Basics; Quick Reference
Tutorial: Create and manage budgets	Tutorial - Create and manage budgets learn.microsoft.com/en-us/azure/cost-management-billing/costs/tutorial-acm-create-budgets	Cost Basics and Safe Habits
Tutorial: Trigger Azure Functions on blob containers using an event subscription	Tutorial: Trigger Azure Functions on blob containers using an event subscription learn.microsoft.com/en-us/azure/azure-functions/functions-event-grid-blob-trigger	Running Your Python App
Upgrade from Azure OpenAI to Foundry	Upgrade from Azure OpenAI to Foundry learn.microsoft.com/en-us/azure/foundry/how-to/upgrade-azure-openai	Models and the Foundry Platform

Resource	Page title at verification / URL	Used in
Use the Azure OpenAI Responses API	Use the Azure OpenAI Responses API learn.microsoft.com/en-us/azure/foundry/openai/how-to/responses	Models and the Foundry Platform
Vector search in Azure Cosmos DB for NoSQL	Cosmos DB vector search learn.microsoft.com/en-us/azure/cosmos-db/nosql/vector-search	Storage and Databases
Vector search overview	Vector search learn.microsoft.com/en-us/azure/search/vector-search-overview	Search and RAG
What is Azure RBAC?	What is Azure RBAC? learn.microsoft.com/en-us/azure/role-based-access-control/overview	Sign-In, Roles and Secrets

Microsoft pricing and tools

Resource	Page title at verification / URL	Used in
Azure AI Search / Foundry IQ pricing	Pricing - Foundry IQ (Azure AI Search) azure.microsoft.com/en-us/pricing/details/search/	Search and RAG
Azure OpenAI pricing	Azure OpenAI Service Pricing azure.microsoft.com/en-us/pricing/details/cognitive-services/openai-service/	Models and the Foundry Platform
Azure pricing calculator	Azure Pricing Calculator azure.microsoft.com/en-us/pricing/calculator/	Cost Basics and Safe Habits
Container Apps pricing page	Azure Container Apps - Pricing azure.microsoft.com/pricing/details/container-apps/	Running Your Python App

SDK references and packages

Resource	Page title at verification / URL	Used in
Azure AI Content Safety Python SDK (PyPI)	PyPI azure-ai-contentsafety 1.0.0 pypi.org/project/azure-ai-contentsafety/	AI Agents and Content Safety
Azure AI Document Intelligence client library for Python	Azure AI Document Intelligence client library for Python learn.microsoft.com/en-us/python/api/overview/azure/ai-documentintelligence-readme	OCR and Document Extraction
Azure AI Search client library for Python	Azure AI Search client library for Python learn.microsoft.com/en-us/python/api/overview/azure/search-documents-readme	Search and RAG
Azure Cosmos DB client library for Python	Azure Cosmos DB client library for Python learn.microsoft.com/en-us/python/api/overview/azure/cosmos-readme	Storage and Databases
Azure Storage Blobs client library for Python	Azure Storage Blobs client library for Python learn.microsoft.com/en-us/python/api/overview/azure/storage-blob-readme	Storage and Databases
azure-ai-documentintelligence (PyPI)	PyPI pypi.org/project/azure-ai-documentintelligence/	OCR and Document Extraction
azure-ai-evaluation package	PyPI pypi.org/project/azure-ai-evaluation/	Search and RAG
azure-search-documents (PyPI)	PyPI pypi.org/project/azure-search-documents/	Search and RAG

Resource	Page title at verification / URL	Used in
DefaultAzureCredential reference	DefaultAzureCredential Class learn.microsoft.com/en-us/python/api/azure-identity/azure.identity.defaultazurecredential	Sign-In, Roles and Secrets
openai-python repository	openai/openai-python github.com/openai/openai-python	Models and the Foundry Platform; Cost Basics and Safe Habits
pgvector project	pgvector/pgvector github.com/pgvector/pgvector	Storage and Databases

Summary: 87 unique external links; 87 verified; 0 not verified.

References: official sources

The guide relies on the following official sources. Third-party sources were used only where noted (certification changes), and are flagged as not fully verified.

- Microsoft Learn (learn.microsoft.com): Microsoft Foundry, Azure AI Search, Foundry Tools, Azure Machine Learning, Azure compute, networking, identity, monitoring, Bicep, Azure CLI reference and Azure SDK for Python reference documentation.
- Microsoft Azure pricing pages and the Azure pricing calculator (azure.microsoft.com).
- Python Package Index (pypi.org) for package names, versions and release dates; official SDK repositories on GitHub.